

Universidade Federal de São Paulo

Implementação do Compilador C-

Nome do aluno: **Bianca Valenciani, RA: 168763**

Professor: Prof. Dr. Luiz Eduardo Galvão Martins

Relatório apresentado à Universidade Federal de São Paulo como parte dos requisitos para aprovação na disciplina de Laboratório de Sistemas Computacionais: Compiladores.

[PREENCHER LOCAL]

Junho de 2026

Sumário

1	Introdução	2
2	O Processador	3
2.1	Diagrama de Blocos do Processador	3
2.2	Componentes do Processador	3
2.3	Conjunto de Instruções	4
2.4	Organização da Memória	4
3	Compilador: Fase de Análise	6
3.1	Modelagem	6
3.1.1	Diagrama de Blocos SysML	6
3.1.2	Diagramas de Atividades SysML	7
3.2	Análise Léxica	8
3.3	Análise Sintática	8
3.4	Análise Semântica	10
4	Compilador: Fase de Síntese	11
4.1	Modelagem	12
4.1.1	Diagrama de Blocos SysML	12
4.1.2	Diagramas de Atividades SysML	13
4.2	Geração do Código Intermediário	14
4.3	Geração do Código Assembly	15
4.4	Geração do Código Executável	16
4.5	Gerenciamento de Memória	17
5	Exemplos	18
5.1	Exemplo 1: GCD/MDC com Recursão	18
5.1.1	Código Fonte	18
5.1.2	Código Intermediário	18
5.1.3	Correspondência: Código Fonte → Código Intermediário	20
5.1.4	Código Assembly	20
5.1.5	Correspondência: Código Intermediário → Assembly	23

5.1.6	Código Executável	23
5.1.7	Resultado de Teste	24
5.2	Exemplo 2: Sort com Vetor como Parâmetro	24
5.2.1	Código Fonte	24
5.2.2	Código Intermediário	25
5.2.3	Código Assembly	28
5.2.4	Código Executável	29
5.2.5	Correspondência: Código Fonte → Código Intermediário	29
5.2.6	Correspondência: Código Intermediário → Assembly	29
5.2.7	Resultado de Teste	29
5.3	Exemplo 3: Teste Completo	30
5.3.1	Código Fonte	30
5.3.2	Código Intermediário	33
5.3.3	Código Assembly	34
5.3.4	Código Executável	35
5.3.5	Correspondência: Código Fonte → Código Intermediário	36
5.3.6	Correspondência: Código Intermediário → Assembly	36
5.3.7	Resultado de Teste	36
6	Conclusão	37
6.1	Dificuldades Encontradas	37
6.2	Destaques	37
6.3	Trabalhos Futuros	38
7	Referências	39
A	Código Completo dos Exemplos	40
A.1	Exemplo 1: GCD	40
A.1.1	Código Intermediário Completo	40
A.1.2	Código Assembly Completo	41
A.1.3	Código Executável (hex) Completo	48
A.2	Exemplo 2: Sort	50
A.2.1	Código Assembly Completo	50
A.2.2	Código Executável (hex) Completo	70
A.3	Exemplo 3: Teste Completo	77
A.3.1	Código Intermediário Completo	77
A.3.2	Código Assembly Completo	85
A.3.3	Código Executável (hex) Completo	144

Capítulo 1

Introdução

Compiladores são sistemas responsáveis por traduzir programas escritos em uma linguagem de alto nível para uma representação executável por uma máquina alvo. Essa tradução envolve etapas de análise e síntese: inicialmente o código fonte é reconhecido, estruturado e validado; em seguida, ele é transformado em código intermediário, assembly e, por fim, código binário.

O projeto desenvolvido na disciplina consiste na implementação de um compilador para a linguagem C-, uma linguagem inspirada em C, porém com um conjunto reduzido de construções. O compilador implementado reconhece declarações de variáveis, vetores, funções, parâmetros, comandos condicionais, comandos de repetição, chamadas de função, entrada e saída, operadores aritméticos e operadores relacionais.

Além da análise do programa fonte, o projeto inclui geração de código intermediário em quádruplas, geração de assembly para um processador RISC-V customizado e geração de código executável em formato hexadecimal. O processador utilizado como alvo foi implementado em Verilog HDL, podendo ser sintetizado no Quartus e simulado no ModelSim.

O fluxo completo do projeto é:

`C- → tokens → AST → tabela de símbolos → quádruplas → assembly →
hexadecimal/Binário → processador`

Este relatório está organizado da seguinte forma. O Capítulo 2 apresenta o processador utilizado como máquina alvo. O Capítulo 3 descreve as fases de análise do compilador. O Capítulo 4 descreve as fases de síntese. O Capítulo 5 apresenta exemplos de uso do compilador. Por fim, o Capítulo 6 discute dificuldades, destaques e trabalhos futuros.

Capítulo 2

O Processador

2.1 Diagrama de Blocos do Processador

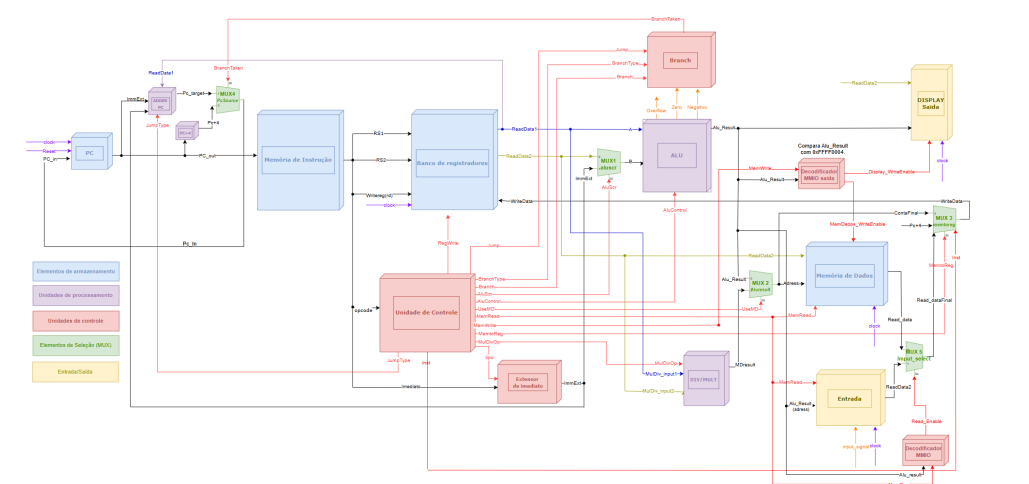


Figura 2.1: Diagrama de blocos do processador RISC-V.

O processador utilizado como alvo do compilador é uma implementação RISC-V customizada em Verilog HDL. Ele possui contador de programa, memória de instruções, unidade de controle, banco de registradores, ULA, unidade de multiplicação/divisão, memória de dados e interface de entrada e saída mapeada em memória.

A execução dos programas gerados pelo compilador ocorre por meio da memória de instruções inicializada com um arquivo hexadecimal. O compilador gera assembly RISC-V e o montador converte esse assembly para o arquivo `program.txt`, utilizado pelo módulo `instruction_memory.v` por meio de `$readmemh`.

2.2 Componentes do Processador

Os principais componentes do processador são:

- **Program Counter (PC):** armazena o endereço da instrução atual.
- **Memória de instruções:** armazena o programa em formato binário hexadecimal.
- **Unidade de controle:** decodifica opcode, funct3 e funct7, gerando sinais de controle.
- **Banco de registradores:** implementa 32 registradores de 32 bits, mantendo x0 constante em zero.
- **ULA:** executa operações aritméticas, lógicas e relacionais.
- **Unidade de multiplicação/divisão:** executa operações da extensão M, como mul, div e rem.
- **Memória de dados:** armazena variáveis globais, vetores e pilha de execução.
- **Interface de entrada e saída:** implementa input() e output() por endereços mapeados em memória.

2.3 Conjunto de Instruções

O processador implementa um subconjunto RV32I com parte da extensão M. O compilador foi projetado para emitir apenas instruções suportadas pela máquina alvo.

Classe	Instruções utilizadas	Finalidade
Tipo R	add, sub, slt, xor, sll, mul, div	Operações entre registradores
Tipo I	addi, slti, xori, lw, jalr	Imediatos, leitura de memória e retorno
Tipo S	sw	Escrita em memória
Tipo B	beq, bne, blt, bge	Desvios condicionais
Tipo J	jal	Chamadas e desvios incondicionais

2.4 Organização da Memória

A memória do sistema foi organizada com 8192 bytes, divididos em palavras de 32 bits. O compilador inicializa o ponteiro de pilha x2 no topo da memória, em torno do endereço 8188, e utiliza crescimento descendente para armazenar variáveis locais, temporários, parâmetros, registrador de retorno e frame pointer.

Registrador	Uso
<code>x0</code>	Constante zero
<code>x1</code>	Endereço de retorno, <code>ra</code>
<code>x2</code>	Ponteiro de pilha, <code>sp</code>
<code>x8</code>	Frame pointer, <code>fp</code>
<code>t0</code> , <code>t1</code> , <code>t2</code>	Temporários usados pelo assembly gerado

Tabela 2.2: Registradores utilizados pela convenção do compilador.

A entrada e a saída são mapeadas em memória:

Endereço	Função
2044	Entrada de dados, usada por <code>input()</code>
2040	Saída de dados, usada por <code>output()</code>

Tabela 2.3: Endereços de I/O mapeados em memória.

Vetores globais são alocados a partir da região inicial da memória de dados. Vetores passados como parâmetro são tratados como endereços-base, permitindo que funções como `sort(int a[])` e `minloc(int a[])` acessem e modifiquem o vetor original.

Capítulo 3

Compilador: Fase de Análise

3.1 Modelagem

3.1.1 Diagrama de Blocos SysML

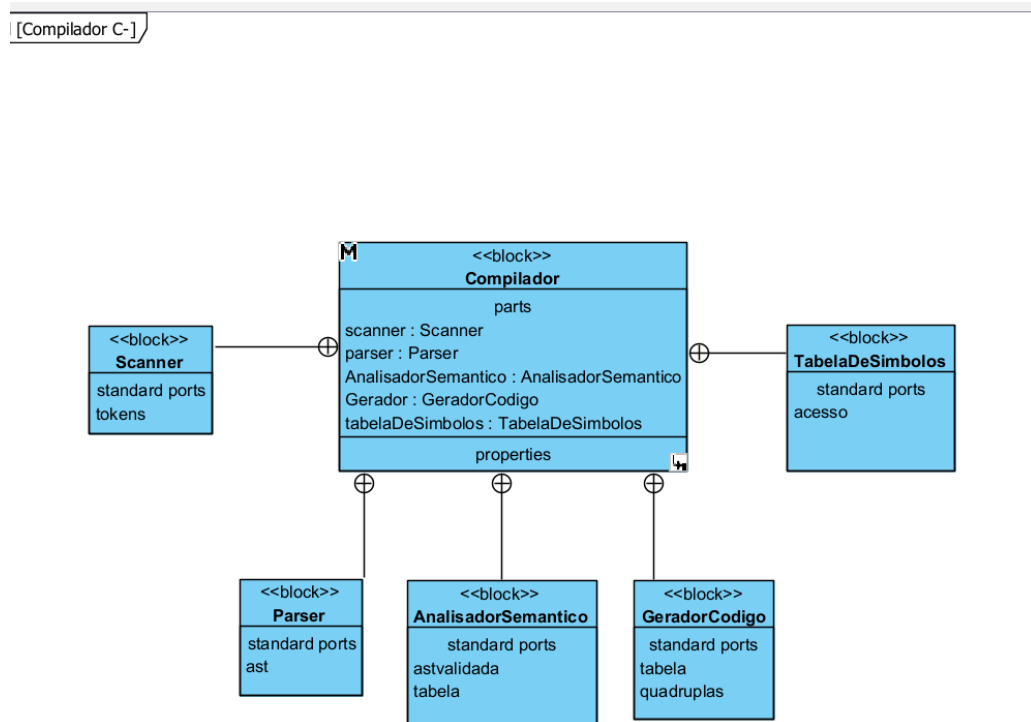


Figura 3.1: Diagrama de blocos SysML do compilador.

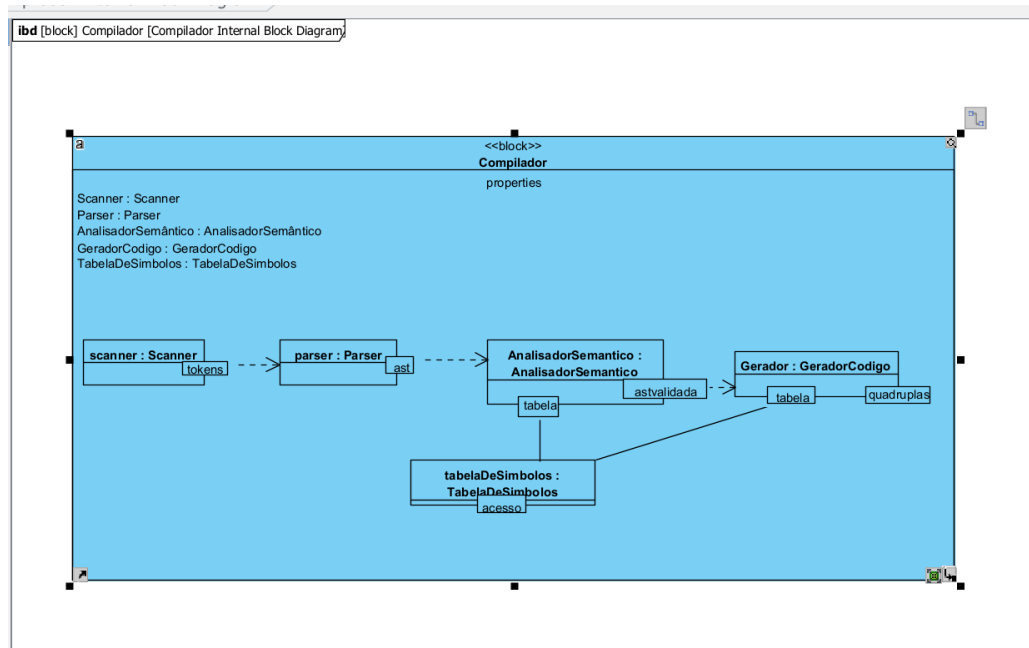


Figura 3.2: Diagrama de blocos interno SysML do compilador.

Os diagramas de blocos acima representam a decomposição do compilador em módulos funcionais. A estrutura geral contém o analisador léxico, o analisador sintático, o analisador semântico, a tabela de símbolos, o gerador de código intermediário, o gerador de assembly e o montador para código executável.

3.1.2 Diagramas de Atividades SysML

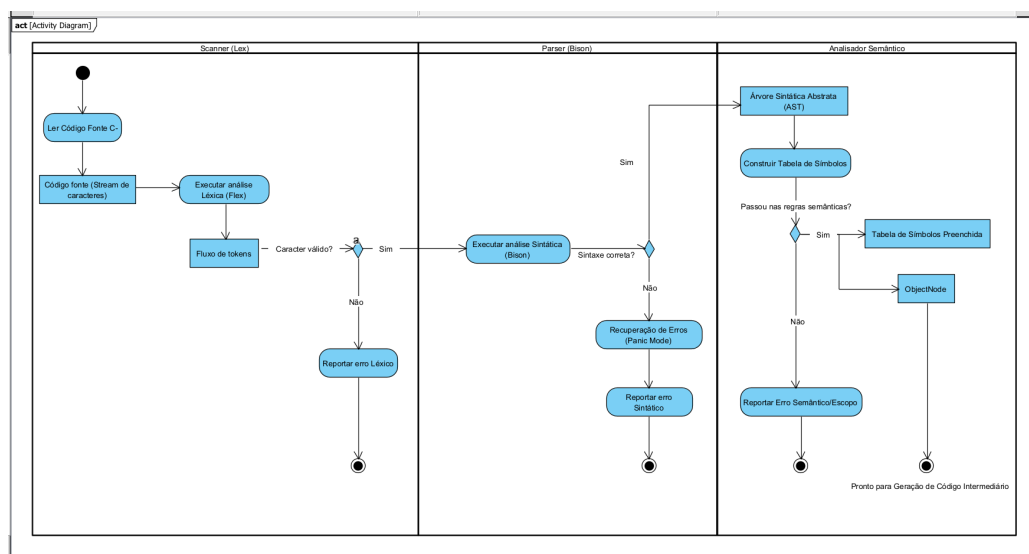


Figura 3.3: Diagrama de atividade da fase de análise.

O fluxo da fase de análise começa com a leitura do arquivo fonte C-. O analisador léxico divide o texto em tokens. Em seguida, o analisador sintático verifica a estrutura

gramatical e constrói a árvore sintática. Por fim, o analisador semântico percorre a árvore, preenche a tabela de símbolos e verifica erros relacionados a escopo, declarações e uso de identificadores.

3.2 Análise Léxica

A análise léxica foi implementada com Flex, no arquivo `src/lexer.1`. Essa fase transforma a sequência de caracteres do programa fonte em tokens. Os principais tokens reconhecidos são:

- palavras-chave: `if`, `else`, `while`, `return`, `int`, `void`;
- identificadores e números inteiros;
- operadores aritméticos: `+`, `-`, `*`, `/`;
- operadores relacionais: `<`, `<=`, `>`, `>=`, `==`, `!=`;
- delimitadores: parênteses, chaves, colchetes, ponto e vírgula e vírgula;
- comentários de bloco no formato `/* ... */`.

Quando um símbolo inválido é encontrado, o analisador léxico informa a linha do erro. Essa etapa simplifica a entrada para o analisador sintático e evita que caracteres desconhecidos cheguem às fases posteriores.

3.3 Análise Sintática

A análise sintática foi implementada com Bison, no arquivo `src/syntax.y`. Ela reconhece a gramática da linguagem C- e constrói uma árvore sintática abstrata. A AST representa a estrutura do programa sem guardar todos os detalhes textuais do código fonte.

Entre as construções reconhecidas estão declarações de variáveis simples, vetores, funções, parâmetros simples e vetoriais, comandos compostos, atribuições, `if`, `if/else`, `while`, `return`, chamadas de função e expressões aritméticas e relacionais.

3.4 Análise Semântica

A análise semântica utiliza a tabela de símbolos para validar o uso dos identificadores. A tabela registra nome, escopo, tipo de identificador, tipo de dado, localização e linhas de ocorrência. Essa estrutura permite detectar erros como uso de variáveis não declaradas, conflitos de declaração e inconsistências relacionadas a funções, variáveis e vetores.

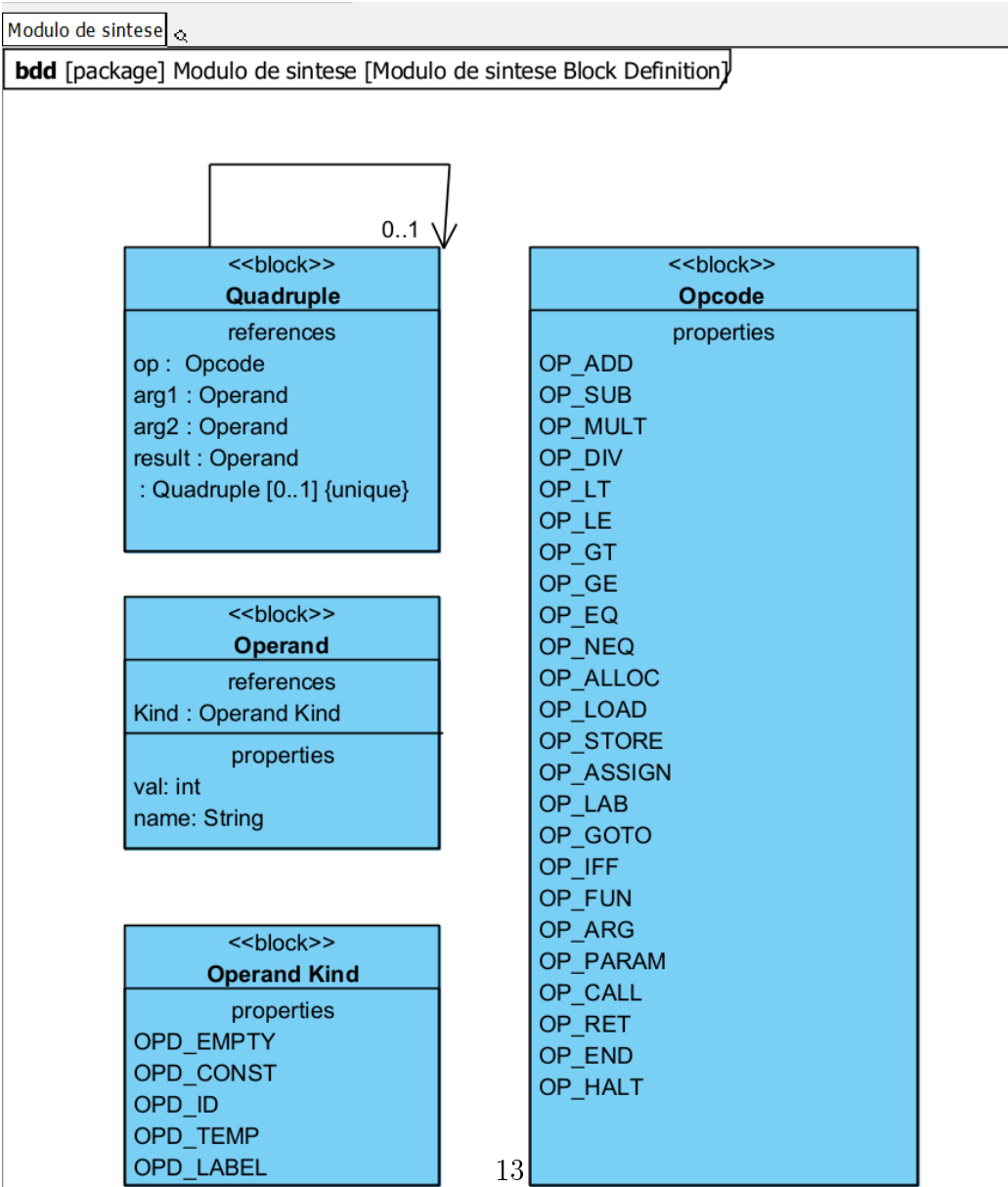
O controle de escopo permite diferenciar símbolos globais, símbolos locais de funções e símbolos declarados dentro de blocos internos. Isso é importante para a linguagem C-, pois funções como `sort` e `minloc` declaram variáveis locais com nomes iguais em escopos diferentes.

Capítulo 4

Compilador: Fase de Síntese

4.1 Modelagem

4.1.1 Diagrama de Blocos SysML



A fase de síntese recebe como entrada a árvore sintática validada e a tabela de símbolos. A partir dessas estruturas, o compilador gera quádruplas, traduz as quádruplas para assembly RISC-V e converte o assembly para código hexadecimal.

4.1.2 Diagramas de Atividades SysML

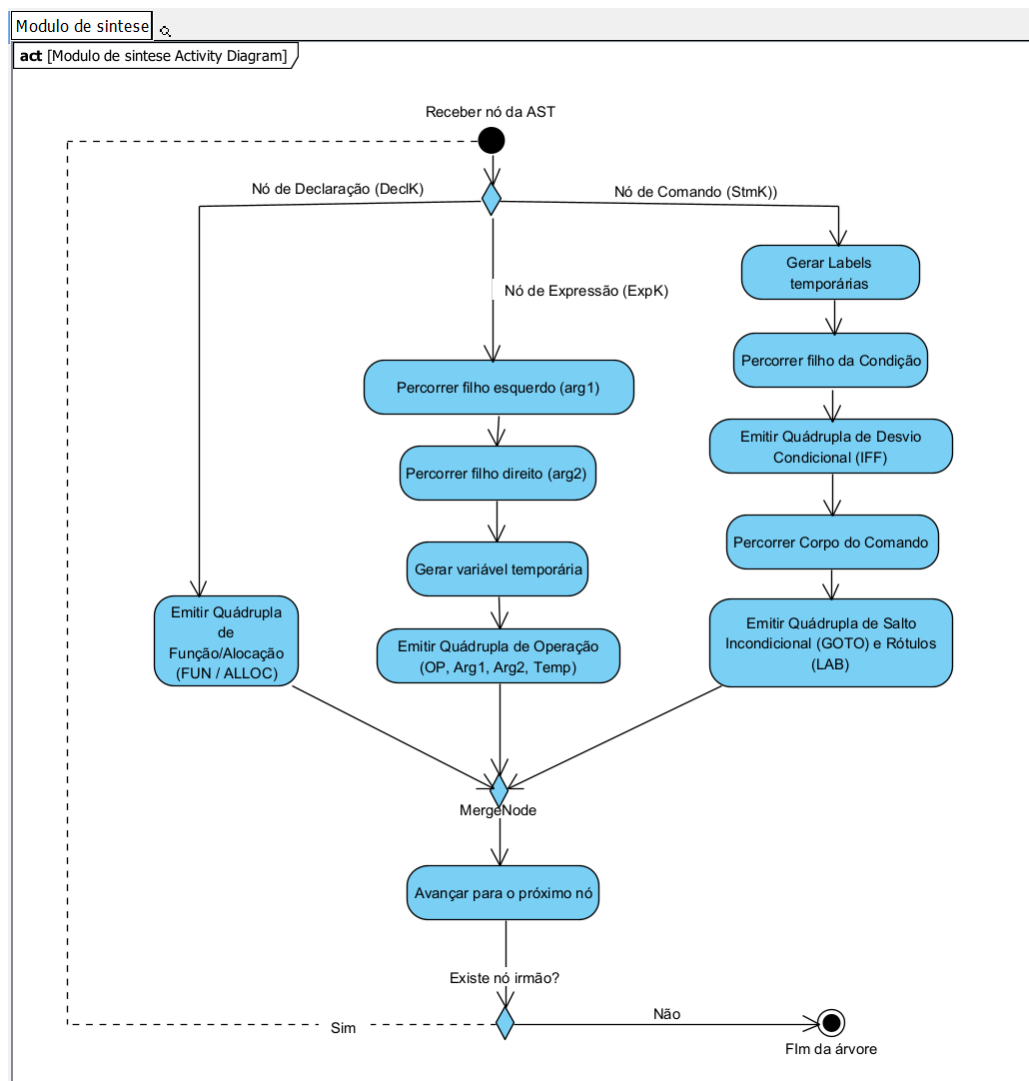


Figura 4.2: Diagrama de atividade da geração de código intermediário.

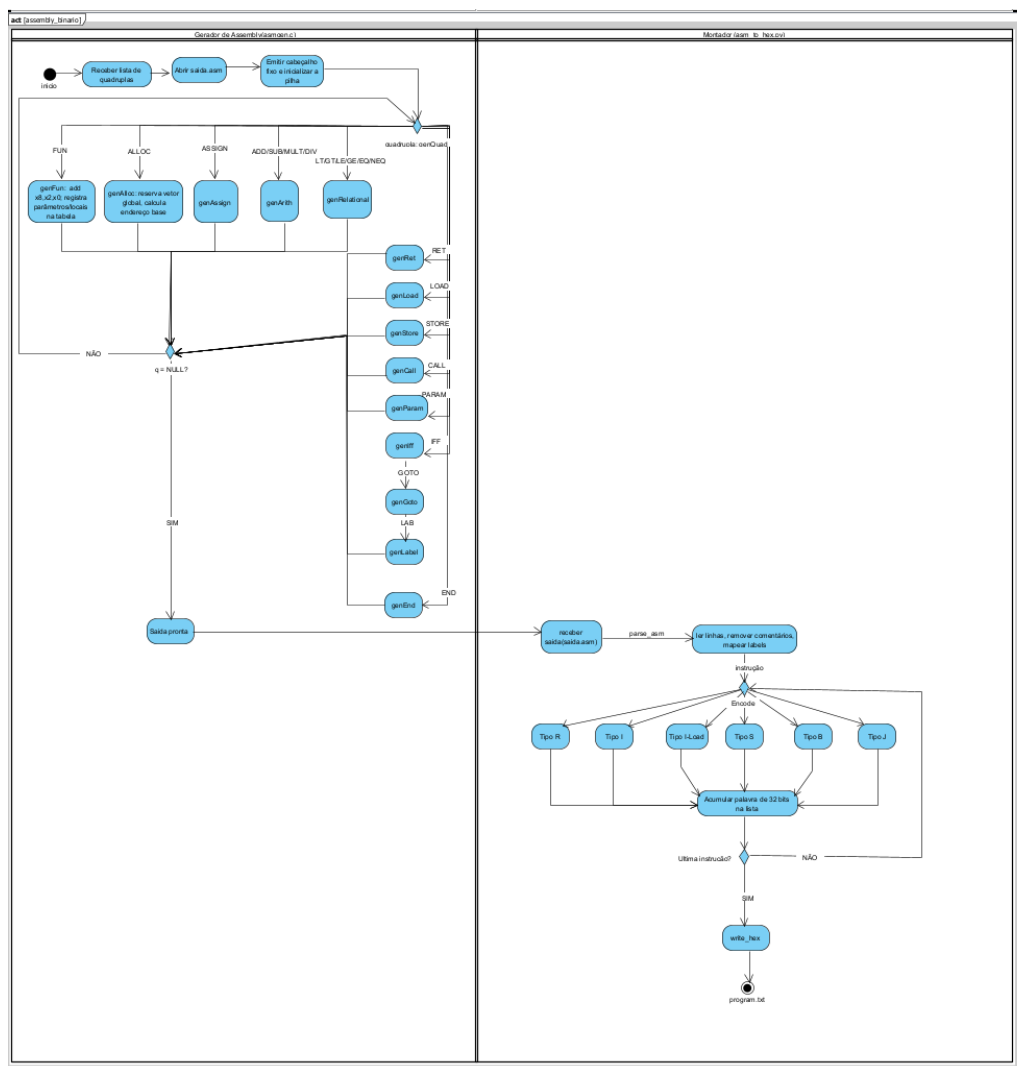


Figura 4.3: Diagrama de atividade da geração de assembly e binário.

4.2 Geração do Código Intermediário

O código intermediário é representado por quádruplas no formato:

(op, arg1, arg2, result)

Cada quádrupla representa uma operação simples. Essa representação facilita a separação entre a análise da linguagem fonte e a geração de código para a arquitetura alvo.

Quádrupla	Significado
(FUN, tipo, nome, -)	Início de função
(END, nome, -, -)	Fim de função
(ASSIGN, val, -, dest)	Atribuição
(ADD, a, b, dest)	Soma
(SUB, a, b, dest)	Subtração

(MULT, a, b, dest)	Multiplicação
(DIV, a, b, dest)	Divisão
(LT, a, b, dest)	Comparação menor que
(LE, a, b, dest)	Comparação menor ou igual
(GT, a, b, dest)	Comparação maior que
(GE, a, b, dest)	Comparação maior ou igual
(EQ, a, b, dest)	Comparação de igualdade
(NEQ, a, b, dest)	Comparação de diferença
(IFF, cond, label, -)	Desvio se a condição for falsa
(GOTO, label, -, -)	Desvio incondicional
(LAB, label, -, -)	Definição de rótulo
(PARAM, arg, -, -)	Passagem de parâmetro
(CALL, func, nargs, dest)	Chamada de função
(RET, val, -, -)	Retorno de função
(LOAD, vet, idx, dest)	Leitura de vetor
(STORE, val, idx, vet)	Escrita em vetor
(ALLOC, vet, tam, -)	Reserva de vetor global

As quádruplas são armazenadas em lista encadeada, permitindo que o gerador de assembly percorra a sequência na mesma ordem em que o programa deve ser executado.

4.3 Geração do Código Assembly

O módulo de geração de assembly está implementado em `src/asmgen.c`. Ele percorre a lista de quádruplas e emite instruções RISC-V compatíveis com o processador desenvolvido.

A geração de assembly utiliza uma convenção simples de chamada de função:

- o registrador `x2` é usado como ponteiro de pilha;
- o registrador `x8` é usado como frame pointer;
- os parâmetros são empilhados pelo chamador;
- antes da chamada, o chamador salva `fp` e `ra`;
- a função chamada cria seu frame local;
- o retorno é salvo no destino indicado pela quádrupla;
- após a chamada, o chamador restaura `ra`, `fp` e remove os parâmetros da pilha.

Exemplo de tradução de chamada:

```
(PARAM, x, -, -)
(PARAM, y, -, -)
(CALL, gcd, 2, _t18)
```

Essa sequência gera o empilhamento dos argumentos, o salvamento de `fp` e `ra`, a instrução `jal` para a função e a restauração do contexto após o retorno.

As operações aritméticas e relacionais são traduzidas para instruções da ULA. Por exemplo, uma quádrupla `ADD` gera `add`, uma quádrupla `SUB` gera `sub`, `MULT` gera `mul` e `DIV` gera `div`. Comparações como `==` e `!=` são implementadas com combinações de `xor`, `slti` e `xori`.

A entrada e a saída são tratadas como funções especiais:

```
x = input();
output(x);
```

No assembly gerado, essas operações acessam os endereços mapeados em memória:

```
lw    t0, 2044(x0) # input
sw    t0, 2040(x0) # output
```

4.4 Geração do Código Executável

A geração do código executável é feita pelo script `asm_to_hex.py`. Esse script monta o assembly gerado pelo compilador e produz um arquivo de texto com uma instrução de 32 bits em hexadecimal por linha (`program.txt`), carregado pelo módulo `instruction_memory.v` por meio de `$readmemh`.

O fluxo é:

```
.\compilador.exe .\testes\gcd.cm
python asm_to_hex.py saida.asm program.txt
```

O arquivo `program.txt` contém uma instrução de 32 bits por linha:

```
7ff00113
7ff10113
7ff10113
7ff10113
100000ef
0000006f
```

4.5 Gerenciamento de Memória

O gerenciamento de memória considera três regiões principais: variáveis e vetores globais, pilha de chamadas e endereços de entrada e saída mapeados em memória.

Vetores globais são reservados a partir do início da memória de dados. Cada posição do vetor ocupa uma palavra de 32 bits, portanto o índice é multiplicado por 4 antes do acesso. No assembly, essa multiplicação é feita por deslocamento:

```
addi t0, x0, 2  
sll  t1, t1, t0
```

Variáveis locais e temporários são alocados em offsets negativos em relação ao frame pointer. Parâmetros ficam em offsets positivos, pois foram empilhados antes da criação do frame da função chamada. Essa organização permite recursão, pois cada chamada possui seu próprio frame na pilha.

Capítulo 5

Exemplos

5.1 Exemplo 1: GCD/MDC com Recursão

5.1.1 Código Fonte

```
int gcd (int u, int v)
{
    if (v == 0) {
        return u;
    }
    else {
        return gcd(v, u - u / v * v);
    }
}

void main(void)
{
    int x; int y;
    x = input();
    y = input();
    output(gcd(x, y));
}
```

5.1.2 Código Intermediário

```
1: (FUN, int, gcd, -)
2: (ASSIGN, v, -, _t1)
3: (ASSIGN, 0, -, _t2)
4: (EQ, _t1, _t2, _t3)
5: (IFF, _t3, L1, -)
```

```
6: (ASSIGN, u, -, _t4)
7: (RET, _t4, -, -)
8: (GOTO, L2, -, -)
9: (LAB, L1, -, -)
10: (ASSIGN, v, -, _t5)
11: (PARAM, _t5, -, -)
12: (ASSIGN, u, -, _t6)
13: (ASSIGN, u, -, _t7)
14: (ASSIGN, v, -, _t8)
15: (DIV, _t7, _t8, _t9)
16: (ASSIGN, v, -, _t10)
17: (MULT, _t9, _t10, _t11)
18: (SUB, _t6, _t11, _t12)
19: (PARAM, _t12, -, -)
20: (CALL, gcd, 2, _t13)
21: (RET, _t13, -, -)
22: (LAB, L2, -, -)
23: (END, gcd, -, -)
24: (FUN, void, main, -)
25: (CALL, input, 0, _t14)
26: (ASSIGN, _t14, -, x)
27: (CALL, input, 0, _t15)
28: (ASSIGN, _t15, -, y)
29: (ASSIGN, x, -, _t16)
30: (PARAM, _t16, -, -)
31: (ASSIGN, y, -, _t17)
32: (PARAM, _t17, -, -)
33: (CALL, gcd, 2, _t18)
34: (PARAM, _t18, -, -)
35: (CALL, output, 1, -)
36: (END, main, -, -)
```

5.1.3 Correspondência: Código Fonte → Código Intermediário

Trecho do código fonte	Quádruplas geradas
int gcd(int u, int v) {	1
if (v == 0)	2-5
return u;	6-8
else (rótulo L1)	9
gcd(v, ...) – 1º argumento	10-11
u - u/v*v – 2º argumento	12-19
gcd(...) – chamada e retorno	20-21
fim do if/else (rótulo L2)	22
} fim de gcd	23
void main(void) {	24
x = input();	25-26
y = input();	27-28
output(gcd(x,y));	29-35
} fim de main	36

5.1.4 Código Assembly

```
# Assembly RISC-V gerado pelo Compilador C-
# Arquitetura: RV32IM customizada
# I via MMIO: endereco 2044
# O via MMIO: endereco 2040
    addi x2, x0, 2047
    addi x2, x2, 2047
    addi x2, x2, 2047
    addi x2, x2, 2047
    jal  x1, main
__halt:
    jal  x0, __halt
# ---- funcao gcd (int) | 2 parametros ----
gcd:
    add  x8, x2, x0
    addi x2, x2, -52
    lw   t0, 8(x8)
    sw   t0, -4(x8)
    addi t0, x0, 0
    sw   t0, -8(x8)
    lw   t0, -4(x8)
    lw   t1, -8(x8)
    xor  t2, t0, t1
```

```

    slti t2, t2, 1
    sw    t2, -12(x8)
    lw    t0, -12(x8)
    beq   t0, x0, L1
    lw    t0, 12(x8)
    sw    t0, -16(x8)
    lw    x10, -16(x8)
    addi  x2, x8, 0
    jalr  x0, x1, 0
    jal   x0, L2
L1:
    lw    t0, 8(x8)
    sw    t0, -20(x8)
    lw    t0, -20(x8)
    addi  x2, x2, -4
    sw    t0, 0(x2)
    lw    t0, 12(x8)
    sw    t0, -24(x8)
    lw    t0, 12(x8)
    sw    t0, -28(x8)
    lw    t0, 8(x8)
    sw    t0, -32(x8)
    lw    t0, -28(x8)
    lw    t1, -32(x8)
    div   t2, t0, t1
    sw    t2, -36(x8)
    lw    t0, 8(x8)
    sw    t0, -40(x8)
    lw    t0, -36(x8)
    lw    t1, -40(x8)
    mul   t2, t0, t1
    sw    t2, -44(x8)
    lw    t0, -24(x8)
    lw    t1, -44(x8)
    sub   t2, t0, t1
    sw    t2, -48(x8)
    lw    t0, -48(x8)
    addi  x2, x2, -4
    sw    t0, 0(x2)
    addi  x2, x2, -4
    sw    x8, 0(x2)

```

```

    addi x2, x2, -4
    sw    x1, 0(x2)
    jal   x1, gcd
    lw    x1, 0(x2)
    lw    x8, 4(x2)
    addi x2, x2, 8
    addi x2, x2, 8
    sw    x10, -52(x8)
    lw    x10, -52(x8)
    addi x2, x8, 0
    jalr x0, x1, 0
L2:
# ---- fim de gcd ----
# ---- funcao main (void) | 0 parametros ----
main:
    add   x8, x2, x0
    addi x2, x2, -28
    lw    t0, 2044(x0)
    sw    t0, -4(x8)
    lw    t0, -4(x8)
    sw    t0, -8(x8)
    lw    t0, 2044(x0)
    sw    t0, -12(x8)
    lw    t0, -12(x8)
    sw    t0, -16(x8)
    lw    t0, -8(x8)
    sw    t0, -20(x8)
    lw    t0, -20(x8)
    addi x2, x2, -4
    sw    t0, 0(x2)
    lw    t0, -16(x8)
    sw    t0, -24(x8)
    lw    t0, -24(x8)
    addi x2, x2, -4
    sw    t0, 0(x2)
    addi x2, x2, -4
    sw    x8, 0(x2)
    addi x2, x2, -4
    sw    x1, 0(x2)
    jal   x1, gcd
    lw    x1, 0(x2)

```



```

lw    x8, 4(x2)
addi  x2, x2, 8
addi  x2, x2, 8
sw    x10, -28(x8)
lw    t0, -28(x8)
addi  x2, x2, -4
sw    t0, 0(x2)
lw    t0, 0(x2)
addi  x2, x2, 4
sw    t0, 2040(x0)
# ---- fim de main ----
addi  x2, x8, 0
jalr  x0, x1, 0

```

5.1.5 Correspondência: Código Intermediário → Assembly

Quádrupla	Assembly correspondente
(EQ, _t1, _t2, _t3)	lw t0,-4(x8); lw t1,-8(x8); xor t2,t0,t1; slti t2,t2,1; sw t2,-12(x8)
(IFF, _t3, L1, -)	lw t0,-12(x8); beq t0,x0,L1
(RET, _t4, -, -)	lw x10,-16(x8); addi x2,x8,0; jalr x0,x1,0
(DIV, _t7, _t8, _t9)	lw t0,-28(x8); lw t1,-32(x8); div t2,t0,t1; sw t2,-36(x8)
(MULT, _t9, _t10, _t11)	lw t0,-36(x8); lw t1,-40(x8); mul t2,t0,t1; sw t2,-44(x8)
(SUB, _t6, _t11, _t12)	lw t0,-24(x8); lw t1,-44(x8); sub t2,t0,t1; sw t2,-48(x8)
(PARAM, _t12, -, -)	lw t0,-48(x8); addi x2,x2,-4; sw t0,0(x2)
(CALL, gcd, 2, _t13)	sw x8,0(x2); sw x1,0(x2); jal x1,gcd; lw x1,0(x2); lw x8,4(x2); sw x10,-52(x8)
(CALL, input, 0, _t14)	lw t0, 2044(x0)
(CALL, output, 1, -)	lw t0,0(x2); addi x2,x2,4; sw t0,2040(x0)

5.1.6 Código Executável

O assembly acima foi convertido pelo `asm_to_hex.py` em `program.txt`, com uma instrução de 32 bits por linha. As primeiras linhas são:

```

7ff00113
7ff10113
7ff10113
7ff10113

```

```
100000ef
0000006f
```

O arquivo completo (106 instruções) está no Apêndice A.1.3.

5.1.7 Resultado de Teste

O assembly gerado para o programa `gcd.cm` foi executado no simulador `simriscv.py`, que interpreta diretamente as instruções RISC-V geradas pelo compilador. Foram testados três casos:

```
Entrada: 48 18 -> Saida: 6
Entrada: 27 9  -> Saida: 9
Entrada: 42 0  -> Saida: 42
```

Os três casos produziram o resultado esperado do algoritmo de Euclides, confirmando a correção da tradução de código fonte para quádruplas, de quádruplas para assembly e da execução do assembly gerado.

5.2 Exemplo 2: Sort com Vetor como Parâmetro

5.2.1 Código Fonte

```
int vet[10];

int minloc(int a[], int low, int high)
{
    int i; int x; int k;
    k = low;
    x = a[low];
    i = low + 1;
    while (i < high) {
        if (a[i] < x) {
            x = a[i];
            k = i;
        }
        i = i + 1;
    }
    return k;
}

void sort(int a[], int low, int high)
```

```

{
    int i; int k;
    i = low;
    while (i < high - 1) {
        int t;
        k = minloc(a, i, high);
        t = a[k];
        a[k] = a[i];
        a[i] = t;
        i = i + 1;
    }
}

```

5.2.2 Código Intermediário

```

1: (ALLOC, vet, 10, -)
2: (FUN, int, minloc, -)
3: (ASSIGN, low, -, _t1)
4: (ASSIGN, _t1, -, k)
5: (ASSIGN, low, -, _t2)
6: (LOAD, a, _t2, _t3)
7: (ASSIGN, _t3, -, x)
8: (ASSIGN, low, -, _t4)
9: (ASSIGN, 1, -, _t5)
10: (ADD, _t4, _t5, _t6)
11: (ASSIGN, _t6, -, i)
12: (LAB, L1, -, -)
13: (ASSIGN, i, -, _t7)
14: (ASSIGN, high, -, _t8)
15: (LT, _t7, _t8, _t9)
16: (IFF, _t9, L2, -)
17: (ASSIGN, i, -, _t10)
18: (LOAD, a, _t10, _t11)
19: (ASSIGN, x, -, _t12)
20: (LT, _t11, _t12, _t13)
21: (IFF, _t13, L3, -)
22: (ASSIGN, i, -, _t14)
23: (LOAD, a, _t14, _t15)
24: (ASSIGN, _t15, -, x)
25: (ASSIGN, i, -, _t16)
26: (ASSIGN, _t16, -, k)

```

```

27: (LAB, L3, -, -)
28: (ASSIGN, i, -, _t17)
29: (ASSIGN, 1, -, _t18)
30: (ADD, _t17, _t18, _t19)
31: (ASSIGN, _t19, -, i)
32: (GOTO, L1, -, -)
33: (LAB, L2, -, -)
34: (ASSIGN, k, -, _t20)
35: (RET, _t20, -, -)
36: (END, minloc, -, -)
37: (FUN, void, sort, -)
38: (ASSIGN, low, -, _t21)
39: (ASSIGN, _t21, -, i)
40: (LAB, L4, -, -)
41: (ASSIGN, i, -, _t22)
42: (ASSIGN, high, -, _t23)
43: (ASSIGN, 1, -, _t24)
44: (SUB, _t23, _t24, _t25)
45: (LT, _t22, _t25, _t26)
46: (IFF, _t26, L5, -)
47: (ASSIGN, a, -, _t27)
48: (PARAM, _t27, -, -)
49: (ASSIGN, i, -, _t28)
50: (PARAM, _t28, -, -)
51: (ASSIGN, high, -, _t29)
52: (PARAM, _t29, -, -)
53: (CALL, minloc, 3, _t30)
54: (ASSIGN, _t30, -, k)
55: (ASSIGN, k, -, _t31)
56: (LOAD, a, _t31, _t32)
57: (ASSIGN, _t32, -, t)
58: (ASSIGN, k, -, _t33)
59: (ASSIGN, i, -, _t34)
60: (LOAD, a, _t34, _t35)
61: (STORE, _t35, _t33, a)
62: (ASSIGN, i, -, _t36)
63: (ASSIGN, t, -, _t37)
64: (STORE, _t37, _t36, a)
65: (ASSIGN, i, -, _t38)
66: (ASSIGN, 1, -, _t39)
67: (ADD, _t38, _t39, _t40)

```

```

68: (ASSIGN, _t40, -, i)
69: (GOTO, L4, -, -)
70: (LAB, L5, -, -)
71: (END, sort, -, -)
72: (FUN, void, main, -)
73: (ASSIGN, 0, -, _t41)
74: (ASSIGN, _t41, -, i)
75: (LAB, L6, -, -)
76: (ASSIGN, i, -, _t42)
77: (ASSIGN, 10, -, _t43)
78: (LT, _t42, _t43, _t44)
79: (IFF, _t44, L7, -)
80: (ASSIGN, i, -, _t45)
81: (CALL, input, 0, _t46)
82: (STORE, _t46, _t45, vet)
83: (ASSIGN, i, -, _t47)
84: (ASSIGN, 1, -, _t48)
85: (ADD, _t47, _t48, _t49)
86: (ASSIGN, _t49, -, i)
87: (GOTO, L6, -, -)
88: (LAB, L7, -, -)
89: (ASSIGN, vet, -, _t50)
90: (PARAM, _t50, -, -)
91: (ASSIGN, 0, -, _t51)
92: (PARAM, _t51, -, -)
93: (ASSIGN, 10, -, _t52)
94: (PARAM, _t52, -, -)
95: (CALL, sort, 3, _t53)
96: (ASSIGN, 0, -, _t54)
97: (ASSIGN, _t54, -, i)
98: (LAB, L8, -, -)
99: (ASSIGN, i, -, _t55)
100: (ASSIGN, 10, -, _t56)
101: (LT, _t55, _t56, _t57)
102: (IFF, _t57, L9, -)
103: (ASSIGN, i, -, _t58)
104: (LOAD, vet, _t58, _t59)
105: (PARAM, _t59, -, -)
106: (CALL, output, 1, -)
107: (ASSIGN, i, -, _t60)
108: (ASSIGN, 1, -, _t61)

```

```

109: (ADD, _t60, _t61, _t62)
110: (ASSIGN, _t62, -, i)
111: (GOTO, L8, -, -)
112: (LAB, L9, -, -)
113: (END, main, -, -)

```

5.2.3 Código Assembly

O assembly completo tem 322 linhas; o trecho abaixo mostra os três padrões principais - leitura de vetor, chamada de função e escrita em vetor. O arquivo completo está disponível no repositório ([saida.asm](#)).

```

# ---- Leitura de vetor: x = a[low] (quadrupla 6: LOAD, a, _t2, _t3) ----
    lw    t1, -12(x8)      # _t2 (indice)
    addi  t0, x0, 2
    sll   t1, t1, t0       # indice * 4
    lw    t0, 16(x8)       # endereco base de 'a' (parametro)
    add   t0, t0, t1       # endereco = base + offset
    lw    t2, 0(t0)        # le a[low]
    sw    t2, -16(x8)      # guarda em _t3

# ---- Chamada de funcao: k = minloc(a, i, high) (quadrupla 53) ----
    addi  x2, x2, -4
    sw    t0, 0(x2)        # PARAM a
    addi  x2, x2, -4
    sw    t0, 0(x2)        # PARAM i
    addi  x2, x2, -4
    sw    t0, 0(x2)        # PARAM high
    addi  x2, x2, -4
    sw    x8, 0(x2)        # salva fp
    addi  x2, x2, -4
    sw    x1, 0(x2)        # salva ra
    jal   x1, minloc
    lw    x1, 0(x2)        # restaura ra
    lw    x8, 4(x2)        # restaura fp
    addi  x2, x2, 8
    addi  x2, x2, 12       # remove os 3 parametros da pilha
    sw    x10, -44(x8)     # captura retorno em _t30

# ---- Escrita em vetor: a[k] = a[i] (quadrupla 61: STORE, _t35, _t33, a) ----
    lw    t1, -64(x8)      # _t33 (indice k)
    addi  t0, x0, 2

```

```

sll  t1, t1, t0
lw   t0, 16(x8)      # endereço base de 'a'
add  t0, t0, t1       # endereço = base + offset
lw   t2, -72(x8)      # _t35 (valor de a[i], calculado antes)
sw   t2, 0(t0)        # a[k] = valor

```

5.2.4 Código Executável

O assembly completo do `sort.cm` gera 300 instruções, convertidas para hexadecimal no Apêndice A.2.2. O assembly completo está no Apêndice A.2.1.

5.2.5 Correspondência: Código Fonte → Código Intermediário

Trecho do código fonte	Quádruplas geradas
<code>int vet[10]; (vetor global)</code>	1
<code>int minloc(int a[], int low, int high) {</code>	2–36
<code>void sort(int a[], int low, int high) {</code>	37–71
<code>void main(void) {</code>	72–113

5.2.6 Correspondência: Código Intermediário → Assembly

Quádrupla	Assembly correspondente
(LOAD, a, _t2, _t3)	<code>lw t1,-12(x8); addi t0,x0,2; sll t1,t1,t0;</code> <code>lw t0,16(x8); add t0,t0,t1; lw t2,0(t0); sw t2,-16(x8)</code>
(CALL, minloc, 3, _t30)	<code>sw x8,0(x2); sw x1,0(x2); jal x1,minloc;</code> <code>lw x1,0(x2); lw x8,4(x2); sw x10,-44(x8)</code>
(STORE, _t35, _t33, a)	<code>lw t1,-64(x8); addi t0,x0,2; sll t1,t1,t0;</code> <code>lw t0,16(x8); add t0,t0,t1; lw t2,-72(x8); sw t2,0(t0)</code>

5.2.7 Resultado de Teste

O assembly gerado para o programa `sort.cm` foi executado no simulador `simriscv.py` com um vetor de 10 elementos:

```

Entrada: 5 3 8 1 9 2 6 4 7 0
Saida:   0 1 2 3 4 5 6 7 8 9

```

O vetor foi ordenado corretamente pelo algoritmo de seleção, confirmando o funcionamento da passagem de vetor como parâmetro (`a[]`) e da recursão entre `sort` e `minloc`.

5.3 Exemplo 3: Teste Completo

O arquivo `teste_completo.cm` valida várias funcionalidades em conjunto: vetores globais, vetores como parâmetro, funções `int` e `void`, chamadas aninhadas, recursão, comandos `while`, `if` e `else`, entrada, saída, operadores aritméticos e operadores relacionais.

5.3.1 Código Fonte

```
int dados[6];
int copia[6];

int igual(int a, int b)
{
    if (a == b) { return 1; }
    else { return 0; }
}

int diferente(int a, int b)
{
    if (a != b) { return 1; }
    else { return 0; }
}

int maximo(int a, int b)
{
    if (a > b) { return a; }
    else { return b; }
}

int distancia(int a, int b)
{
    if (a >= b) { return a - b; }
    else { return b - a; }
}

int fib(int n)
{
    if (n <= 1) { return n; }
    else { return fib(n - 1) + fib(n - 2); }
}

int resto(int a, int b)
```



```

{
    int q; int r;
    q = a / b;
    r = a - q * b;
    return r;
}

int mdc(int a, int b)
{
    int r;
    while (b != 0) {
        r = resto(a, b);
        a = b;
        b = r;
    }
    return a;
}

void ler(int v[], int n)
{
    int i;
    i = 0;
    while (i < n) {
        v[i] = input();
        i = i + 1;
    }
}

void copiareverso(int origem[], int destino[], int n)
{
    int i; int j;
    i = 0;
    while (i < n) {
        j = n - 1 - i;
        destino[i] = origem[j];
        i = i + 1;
    }
}

int soma(int v[], int n)
{

```

```

    int i; int s;
    i = 0;
    s = 0;
    while (i < n) {
        s = s + v[i];
        i = i + 1;
    }
    return s;
}

int produto(int v[], int n)
{
    int i; int p;
    i = 0;
    p = 1;
    while (i < n) {
        if (v[i] > 0) { p = p * v[i]; }
        else { p = p * 1; }
        i = i + 1;
    }
    return p;
}

void main(void)
{
    int s; int p; int m; int d; int f; int eq; int ne; int r; int g; int
    primeiro; int ultimo;

    ler(dados, 6);
    copiareverso(dados, copia, 6);

    s = soma(dados, 6);
    p = produto(dados, 6);
    m = maximo(dados[0], dados[5]);
    d = distancia(dados[1], dados[4]);
    f = fib(6);
    eq = igual(dados[2], dados[3]);
    ne = diferente(dados[2], dados[3]);
    r = resto(s, 5);
    g = mdc(s, p);
    primeiro = copia[0];

```

```
ultimo = copia[5];

output(s); output(p); output(m); output(d); output(f);
output(eq); output(ne); output(r); output(g);
output(primeiro); output(ultimo);
}
```

5.3.2 Código Intermediário

O programa gera 332 quádruplas, uma para cada função (igual, diferente, maximo, distancia, fib, resto, mdc, ler, copiareverso, soma, produto e main). O trecho abaixo mostra a função fib, que ilustra a recursão dupla:

```
59: (FUN, int, fib, -)
60: (ASSIGN, n, -, _t25)
61: (ASSIGN, 1, -, _t26)
62: (LE, _t25, _t26, _t27)
63: (IFF, _t27, L9, -)
64: (ASSIGN, n, -, _t28)
65: (RET, _t28, -, -)
66: (GOTO, L10, -, -)
67: (LAB, L9, -, -)
68: (ASSIGN, n, -, _t29)
69: (ASSIGN, 1, -, _t30)
70: (SUB, _t29, _t30, _t31)
71: (PARAM, _t31, -, -)
72: (CALL, fib, 1, _t32)
73: (ASSIGN, n, -, _t33)
74: (ASSIGN, 2, -, _t34)
75: (SUB, _t33, _t34, _t35)
76: (PARAM, _t35, -, -)
77: (CALL, fib, 1, _t36)
78: (ADD, _t32, _t36, _t37)
79: (RET, _t37, -, -)
80: (LAB, L10, -, -)
81: (END, fib, -, -)
```

O conteúdo completo (332 quádruplas, todas as funções) está no Apêndice A.3.1.

5.3.3 Código Assembly

Trecho correspondente à função `fib` (mostra as duas chamadas recursivas e a soma dos retornos):

```
fib:
    add  x8, x2, x0
    addi x2, x2, -52
    lw   t0, 8(x8)
    sw   t0, -4(x8)
    addi t0, x0, 1
    sw   t0, -8(x8)
    lw   t0, -4(x8)
    lw   t1, -8(x8)
    slt  t2, t1, t0
    xori t2, t2, 1
    sw   t2, -12(x8)
    lw   t0, -12(x8)
    beq  t0, x0, L9
    lw   t0, 8(x8)
    sw   t0, -16(x8)
    lw   x10, -16(x8)
    addi x2, x8, 0
    jalr x0, x1, 0
    jal  x0, L10
L9:
    lw   t0, 8(x8)
    sw   t0, -20(x8)
    addi t0, x0, 1
    sw   t0, -24(x8)
    lw   t0, -20(x8)
    lw   t1, -24(x8)
    sub  t2, t0, t1
    sw   t2, -28(x8)
    lw   t0, -28(x8)
    addi x2, x2, -4
    sw   t0, 0(x2)
    addi x2, x2, -4
    sw   x8, 0(x2)
    addi x2, x2, -4
    sw   x1, 0(x2)
    jal  x1, fib
    lw   x1, 0(x2)
```

```

lw    x8, 4(x2)
addi  x2, x2, 8
addi  x2, x2, 4
sw    x10, -32(x8)
lw    t0, 8(x8)
sw    t0, -36(x8)
addi  t0, x0, 2
sw    t0, -40(x8)
lw    t0, -36(x8)
lw    t1, -40(x8)
sub   t2, t0, t1
sw    t2, -44(x8)
lw    t0, -44(x8)
addi  x2, x2, -4
sw    t0, 0(x2)
addi  x2, x2, -4
sw    x8, 0(x2)
addi  x2, x2, -4
sw    x1, 0(x2)
jal   x1, fib
lw    x1, 0(x2)
lw    x8, 4(x2)
addi  x2, x2, 8
addi  x2, x2, 4
sw    x10, -48(x8)
lw    t0, -32(x8)
lw    t1, -48(x8)
add   t2, t0, t1
sw    t2, -52(x8)
lw    x10, -52(x8)
addi  x2, x8, 0
jalr  x0, x1, 0
L10:
# ---- fim de fib ----

```

O assembly completo (975 linhas, 12 funções) está no Apêndice A.3.2.

5.3.4 Código Executável

O hexadecimal completo (928 instruções) está no Apêndice A.3.3.

5.3.5 Correspondência: Código Fonte → Código Intermediário

Função	Quádruplas
int dados[6]; int copia[6];	1-2
igual	3-15
diferente	16-28
maximo	29-41
distancia	42-58
fib	59-81
resto	82-95
mdc	96-116
ler	117-134
copiareverso	135-159
soma	160-183
produto	184-219
main	220-332

5.3.6 Correspondência: Código Intermediário → Assembly

Quádrupla	Assembly correspondente
(CALL, fib, 1, _t32) (1ª chamada, fib(n-1))	sw x8,0(x2); sw x1,0(x2); jal x1,fib; lw x1,0(x2); lw x8,4(x2); sw x10,-32(x8)
(CALL, fib, 1, _t36) (2ª chamada, fib(n-2))	sw x8,0(x2); sw x1,0(x2); jal x1,fib; lw x1,0(x2); lw x8,4(x2); sw x10,-48(x8)
(ADD, _t32, _t36, _t37)	lw t0,-32(x8); lw t1,-48(x8); add t2,t0,t1; sw t2,-52(x8)

5.3.7 Resultado de Teste

O assembly gerado para o programa teste_completo.cm foi executado no simulador simriscv.py:

Entrada:

5 3 8 1 9 2

Saida:

28 2160 5 6 8 0 1 3 4 2 5

As 11 saídas correspondem aos 11 valores calculados em main (soma, produto, máximo, distância, Fibonacci, igualdade, diferença, resto, MDC e os elementos invertidos do vetor), confirmando o funcionamento conjunto de vetores globais, vetores como parâmetro, recursão e chamadas aninhadas.

Capítulo 6

Conclusão

6.1 Dificuldades Encontradas

Uma das principais dificuldades do projeto foi manter a consistência entre as diferentes representações do programa: árvore sintática, tabela de símbolos, quádruplas, assembly e código binário. Cada fase depende da fase anterior, de modo que erros pequenos em escopo, temporários ou offsets de memória podem aparecer apenas durante a execução final.

Outra dificuldade importante foi a implementação de chamadas de função e recursão. Para que chamadas recursivas funcionem corretamente, é necessário salvar e restaurar o endereço de retorno, preservar o frame pointer e organizar parâmetros e variáveis locais de forma consistente na pilha.

Também houve desafios na integração com o processador. A memória de dados e os sinais de controle precisam seguir a mesma semântica assumida pelo código gerado. As instruções geradas pelo compilador devem estar exatamente dentro do subconjunto implementado no hardware. Ocoreram também problemas de driver entre meu computador e o FPGA.

6.2 Destaques

O projeto implementa um fluxo completo de compilação e execução. Entre os principais destaques estão a geração de tabela de símbolos com escopos, geração de árvore sintática, geração de código intermediário em quádruplas, suporte a funções, parâmetros e recursão, suporte a vetores globais e vetores como parâmetros, geração de assembly RISC-V, geração de código hexadecimal e validação em simulação HDL no ModelSim.

6.3 Trabalhos Futuros

Como trabalhos futuros, podem ser implementadas melhorias como suporte a mais tipos de dados, mensagens de erro semântico mais detalhadas, otimizações simples sobre as quádruplas, melhor alocação de registradores, suporte a mais instruções RISC-V, adaptação do processador para memória síncrona com pipeline ou múltiplos ciclos e automação completa da execução na placa FPGA. No próximo laboratório, será desenvolvido um Sistema Operacional que permitirá execução de múltiplos programas.

Capítulo 7

Referências

1. LOUDEN, Kenneth C. *Compiler Construction: Principles and Practice*. Boston: PWS Publishing, 1997.
2. MARTINS, Luiz Eduardo Galvão. Slides e aulas da disciplina Laboratório de Sistemas Computacionais: Compiladores. Universidade Federal de São Paulo, 1º semestre de 2026.
3. CONTRERAS, Rodrigo Conalgo. Aulas e materiais de apoio sobre compiladores, análise sintática, análise semântica e geração de código.
4. Relatório do processador RISC-V desenvolvido no projeto. Arquivo `Relatório Final/RISCV_relatórioFinal.pdf`.
5. The RISC-V Instruction Set Manual. Referência utilizada para codificação das instruções RV32I/RV32M empregadas no processador e no montador.

Apêndice A

Código Completo dos Exemplos

A.1 Exemplo 1: GCD

A.1.1 Código Intermediário Completo

```
1: (FUN, int, gcd, -)
2: (ASSIGN, v, -, _t1)
3: (ASSIGN, 0, -, _t2)
4: (EQ, _t1, _t2, _t3)
5: (IFF, _t3, L1, -)
6: (ASSIGN, u, -, _t4)
7: (RET, _t4, -, -)
8: (GOTO, L2, -, -)
9: (LAB, L1, -, -)
10: (ASSIGN, v, -, _t5)
11: (PARAM, _t5, -, -)
12: (ASSIGN, u, -, _t6)
13: (ASSIGN, u, -, _t7)
14: (ASSIGN, v, -, _t8)
15: (DIV, _t7, _t8, _t9)
16: (ASSIGN, v, -, _t10)
17: (MULT, _t9, _t10, _t11)
18: (SUB, _t6, _t11, _t12)
19: (PARAM, _t12, -, -)
20: (CALL, gcd, 2, _t13)
21: (RET, _t13, -, -)
22: (LAB, L2, -, -)
23: (END, gcd, -, -)
24: (FUN, void, main, -)
25: (CALL, input, 0, _t14)
```

```

26: (ASSIGN, _t14, -, x)
27: (CALL, input, 0, _t15)
28: (ASSIGN, _t15, -, y)
29: (ASSIGN, x, -, _t16)
30: (PARAM, _t16, -, -)
31: (ASSIGN, y, -, _t17)
32: (PARAM, _t17, -, -)
33: (CALL, gcd, 2, _t18)
34: (PARAM, _t18, -, -)
35: (CALL, output, 1, -)
36: (END, main, -, -)

```

A.1.2 Código Assembly Completo

```

# Assembly RISC-V gerado pelo Compilador C-
# Arquitetura: RV32IM customizada
# I via MMIO: endereco 2044
# O via MMIO: endereco 2040

    addi x2, x0, 2047
    addi x2, x2, 2047
    addi x2, x2, 2047
    addi x2, x2, 2047
    jal  x1, main
__halt:
    jal  x0, __halt

# ---- funcao gcd (int) | 2 parametros ----
gcd:
    add  x8, x2, x0
    addi x2, x2, -52

    lw   t0, 8(x8)

```

```
sw    t0, -4(x8)

addi  t0, x0, 0

sw    t0, -8(x8)

lw    t0, -4(x8)

lw    t1, -8(x8)

xor   t2, t0, t1
slti  t2, t2, 1

sw    t2, -12(x8)

lw    t0, -12(x8)

beq   t0, x0, L1

lw    t0, 12(x8)

sw    t0, -16(x8)

lw    x10, -16(x8)

addi  x2, x8, 0
jalr  x0, x1, 0
```

```
jal x0, L2
```

L1:

```
lw t0, 8(x8)
```

```
sw t0, -20(x8)
```

```
lw t0, -20(x8)
```

```
addi x2, x2, -4
```

```
sw t0, 0(x2)
```

```
lw t0, 12(x8)
```

```
sw t0, -24(x8)
```

```
lw t0, 12(x8)
```

```
sw t0, -28(x8)
```

```
lw t0, 8(x8)
```

```
sw t0, -32(x8)
```

```
lw t0, -28(x8)
```

```
lw t1, -32(x8)
```

```
div  t2, t0, t1
```

```
sw   t2, -36(x8)
```

```
lw   t0, 8(x8)
```

```
sw   t0, -40(x8)
```

```
lw   t0, -36(x8)
```

```
lw   t1, -40(x8)
```

```
mul  t2, t0, t1
```

```
sw   t2, -44(x8)
```

```
lw   t0, -24(x8)
```

```
lw   t1, -44(x8)
```

```
sub  t2, t0, t1
```

```
sw   t2, -48(x8)
```

```
lw   t0, -48(x8)
```

```
addi x2, x2, -4
sw    t0, 0(x2)
```

```
addi x2, x2, -4
sw    x8, 0(x2)
addi x2, x2, -4
sw    x1, 0(x2)
jal   x1, gcd
lw    x1, 0(x2)
lw    x8, 4(x2)
addi x2, x2, 8
```

```
addi x2, x2, 8
sw    x10, -52(x8)
```

```
lw    x10, -52(x8)
```

```
addi x2, x8, 0
jalr x0, x1, 0
```

L2:

```
# ---- fim de gcd ----
addi x2, x8, 0
jalr x0, x1, 0
```

```
# ---- funcao main (void) | 0 parametros ----
main:
add   x8, x2, x0
addi x2, x2, -28
```

```
lw    t0, 2044(x0)
sw    t0, -4(x8)
```

```
lw    t0, -4(x8)
```

```
sw    t0, -8(x8)
```

```
lw    t0, 2044(x0)
sw    t0, -12(x8)
```

```
lw    t0, -12(x8)
```

```
sw    t0, -16(x8)
```

```
lw    t0, -8(x8)
```

```
sw    t0, -20(x8)
```

```
lw    t0, -20(x8)
```

```
addi x2, x2, -4
sw    t0, 0(x2)
```

```
lw    t0, -16(x8)
```



```
sw    t0, -24(x8)
```

```
lw    t0, -24(x8)
```

```
addi x2, x2, -4
```

```
sw    t0, 0(x2)
```

```
addi x2, x2, -4
```

```
sw    x8, 0(x2)
```

```
addi x2, x2, -4
```

```
sw    x1, 0(x2)
```

```
jal   x1, gcd
```

```
lw    x1, 0(x2)
```

```
lw    x8, 4(x2)
```

```
addi x2, x2, 8
```

```
addi x2, x2, 8
```

```
sw    x10, -28(x8)
```

```
lw    t0, -28(x8)
```

```
addi x2, x2, -4
```

```
sw    t0, 0(x2)
```

```
lw    t0, 0(x2)
```

```
addi x2, x2, 4
```

```
sw    t0, 2040(x0)
```

```
# ---- fim de main ----
```

```
addi x2, x8, 0
```

```
jalr x0, x1, 0
```

A.1.3 Código Executável (hex) Completo

```
7ff00113
7ff10113
7ff10113
7ff10113
100000ef
0000006f
00010433
fcc10113
00842283
fe542e23
00000293
fe542c23
ffc42283
ff842303
0062c3b3
0013a393
fe742a23
ff442283
00028e63
00c42283
fe542823
ff042503
00040113
00008067
0a80006f
00842283
fe542623
fec42283
ffc10113
00512023
00c42283
fe542423
00c42283
fe542223
00842283
fe542023
fe442283
```

fe042303
0262c3b3
fc742e23
00842283
fc542c23
fdc42283
fd842303
026283b3
fc742a23
fe842283
fd442303
406283b3
fc742823
fd042283
ffc10113
00512023
ffc10113
00812023
ffc10113
00112023
f35ff0ef
00012083
00412403
00810113
00810113
fca42623
fcc42503
00040113
00008067
00040113
00008067
00010433
fe410113
7fc02283
fe542e23
ffc42283
fe542c23
7fc02283
fe542a23
ff442283
fe542823

```
ff842283
fe542623
fec42283
ffc10113
00512023
ff042283
fe542423
fe842283
ffc10113
00512023
ffc10113
00812023
ffc10113
00112023
ea9ff0ef
00012083
00412403
00810113
00810113
fea42223
fe442283
ffc10113
00512023
00012283
00410113
7e502c23
00040113
00008067
```

A.2 Exemplo 2: Sort

A.2.1 Código Assembly Completo

```
# Assembly RISC-V gerado pelo Compilador C-
# Arquitetura: RV32IM customizada
# I via MMIO: endereço 2044
# O via MMIO: endereço 2040

    addi x2, x0, 2047
    addi x2, x2, 2047
    addi x2, x2, 2047
```

```

    addi x2, x2, 2047
    jal  x1, main
__halt:
    jal  x0, __halt

# vetor global 'vet[10]' reservado no endereco 0


# ---- funcao minloc (int) | 3 parametros ----
minloc:
    add  x8, x2, x0
    addi x2, x2, -92

    lw   t0, 12(x8)

    sw   t0, -4(x8)

    lw   t0, -4(x8)

    sw   t0, -8(x8)

    lw   t0, 12(x8)

    sw   t0, -12(x8)

    lw   t1, -12(x8)

    addi t0, x0, 2

```

```
sll  t1, t1, t0
lw   t0, 16(x8)
add  t0, t0, t1
lw   t2, 0(t0)
sw   t2, -16(x8)

lw   t0, -16(x8)

sw   t0, -20(x8)

lw   t0, 12(x8)

sw   t0, -24(x8)

addi t0, x0, 1

sw   t0, -28(x8)

lw   t0, -24(x8)

lw   t1, -28(x8)

add  t2, t0, t1

sw   t2, -32(x8)

lw   t0, -32(x8)

sw   t0, -36(x8)
```

L1:

lw t0, -36(x8)

sw t0, -40(x8)

lw t0, 8(x8)

sw t0, -44(x8)

lw t0, -40(x8)

lw t1, -44(x8)

slt t2, t0, t1

sw t2, -48(x8)

lw t0, -48(x8)

beq t0, x0, L2

lw t0, -36(x8)

sw t0, -52(x8)

```
lw    t1, -52(x8)
```

```
addi  t0, x0, 2
```

```
sll   t1, t1, t0
```

```
lw    t0, 16(x8)
```

```
add   t0, t0, t1
```

```
lw    t2, 0(t0)
```

```
sw    t2, -56(x8)
```

```
lw    t0, -20(x8)
```

```
sw    t0, -60(x8)
```

```
lw    t0, -56(x8)
```

```
lw    t1, -60(x8)
```

```
slt   t2, t0, t1
```

```
sw    t2, -64(x8)
```

```
lw    t0, -64(x8)
```

```
beq   t0, x0, L3
```

```
lw    t0, -36(x8)
```

```
sw    t0, -68(x8)
```



```
lw    t1, -68(x8)
```

```
addi  t0, x0, 2
```

```
sll   t1, t1, t0
```

```
lw    t0, 16(x8)
```

```
add   t0, t0, t1
```

```
lw    t2, 0(t0)
```

```
sw    t2, -72(x8)
```

```
lw    t0, -72(x8)
```

```
sw    t0, -20(x8)
```

```
lw    t0, -36(x8)
```

```
sw    t0, -76(x8)
```

```
lw    t0, -76(x8)
```

```
sw    t0, -8(x8)
```

L3:

```
lw    t0, -36(x8)
```

```
sw    t0, -80(x8)
```

```
addi  t0, x0, 1
```

```
sw    t0, -84(x8)
```

```
lw    t0, -80(x8)
```

```
lw    t1, -84(x8)
```

```
add   t2, t0, t1
```

```
sw    t2, -88(x8)
```

```
lw    t0, -88(x8)
```

```
sw    t0, -36(x8)
```

```
jal   x0, L1
```

L2:

```
lw    t0, -8(x8)
```

```
sw    t0, -92(x8)
```

```
lw    x10, -92(x8)
```

```
addi  x2, x8, 0
```

```
jalr  x0, x1, 0
```

```
# ---- fim de minloc ----
```

```
addi x2, x8, 0
jalr x0, x1, 0
```

```
# ---- funcao sort (void) | 3 parametros ----
```

```
sort:
```

```
    add  x8, x2, x0
    addi x2, x2, -92
```

```
    lw   t0, 12(x8)
```

```
    sw   t0, -4(x8)
```

```
    lw   t0, -4(x8)
```

```
    sw   t0, -8(x8)
```

```
L4:
```

```
    lw   t0, -8(x8)
```

```
    sw   t0, -12(x8)
```

```
    lw   t0, 8(x8)
```

```
sw    t0, -16(x8)
```

```
addi  t0, x0, 1
```

```
sw    t0, -20(x8)
```

```
lw    t0, -16(x8)
```

```
lw    t1, -20(x8)
```

```
sub   t2, t0, t1
```

```
sw    t2, -24(x8)
```

```
lw    t0, -12(x8)
```

```
lw    t1, -24(x8)
```

```
slt   t2, t0, t1
```

```
sw    t2, -28(x8)
```

```
lw    t0, -28(x8)
```

```
beq   t0, x0, L5
```

```
lw    t0, 16(x8)
```

```
sw    t0, -32(x8)
```

```
lw    t0, -32(x8)
```

```
addi x2, x2, -4  
sw    t0, 0(x2)
```

```
lw    t0, -8(x8)
```

```
sw    t0, -36(x8)
```

```
lw    t0, -36(x8)
```

```
addi x2, x2, -4  
sw    t0, 0(x2)
```

```
lw    t0, 8(x8)
```

```
sw    t0, -40(x8)
```

```
lw    t0, -40(x8)
```

```
addi x2, x2, -4  
sw    t0, 0(x2)
```

```
addi x2, x2, -4  
sw    x8, 0(x2)  
addi x2, x2, -4
```

```

sw    x1, 0(x2)
jal   x1, minloc
lw    x1, 0(x2)
lw    x8, 4(x2)
addi  x2, x2, 8

addi  x2, x2, 12
sw    x10, -44(x8)

lw    t0, -44(x8)

sw    t0, -48(x8)

lw    t0, -48(x8)

sw    t0, -52(x8)

lw    t1, -52(x8)

addi  t0, x0, 2
sll   t1, t1, t0
lw    t0, 16(x8)
add   t0, t0, t1
lw    t2, 0(t0)
sw    t2, -56(x8)

lw    t0, -56(x8)

sw    t0, -60(x8)

```

```
lw    t0, -48(x8)
```

```
sw    t0, -64(x8)
```

```
lw    t0, -8(x8)
```

```
sw    t0, -68(x8)
```

```
lw    t1, -68(x8)
```

```
addi  t0, x0, 2
```

```
sll   t1, t1, t0
```

```
lw    t0, 16(x8)
```

```
add   t0, t0, t1
```

```
lw    t2, 0(t0)
```

```
sw    t2, -72(x8)
```

```
lw    t1, -64(x8)
```

```
addi  t0, x0, 2
```

```
sll   t1, t1, t0
```

```
lw    t0, 16(x8)
```

```
add   t0, t0, t1
```

```
lw    t2, -72(x8)
```

```
sw    t2, 0(t0)
```

```
lw    t0, -8(x8)
```

```
sw    t0, -76(x8)
```

```
lw    t0, -60(x8)
```

```
sw    t0, -80(x8)
```

```
lw    t1, -76(x8)
```

```
addi  t0, x0, 2
```

```
sll   t1, t1, t0
```

```
lw    t0, 16(x8)
```

```
add   t0, t0, t1
```

```
lw    t2, -80(x8)
```

```
sw    t2, 0(t0)
```

```
lw    t0, -8(x8)
```

```
sw    t0, -84(x8)
```

```
addi  t0, x0, 1
```

```
sw    t0, -88(x8)
```

```
lw    t0, -84(x8)
```

```
lw    t1, -88(x8)
```

```
add   t2, t0, t1
```

```
sw    t2, -92(x8)
```



```

    lw    t0, -92(x8)

    sw    t0, -8(x8)

    jal   x0, L4

L5:

# ---- fim de sort ----
    addi  x2, x8, 0
    jalr  x0, x1, 0

# ---- funcao main (void) | 0 parametros ----
main:
    add   x8, x2, x0
    addi  x2, x2, -92

    addi  t0, x0, 0

    sw    t0, -4(x8)

    lw    t0, -4(x8)

    sw    t0, -8(x8)

L6:

    lw    t0, -8(x8)

```

```
sw    t0, -12(x8)
```

```
addi  t0, x0, 10
```

```
sw    t0, -16(x8)
```

```
lw    t0, -12(x8)
```

```
lw    t1, -16(x8)
```

```
slt   t2, t0, t1
```

```
sw    t2, -20(x8)
```

```
lw    t0, -20(x8)
```

```
beq   t0, x0, L7
```

```
lw    t0, -8(x8)
```

```
sw    t0, -24(x8)
```

```
lw    t0, 2044(x0)
```

```
sw    t0, -28(x8)
```

```
lw    t1, -24(x8)
```

```
addi  t0, x0, 2
```

```
sll   t1, t1, t0
```

```
addi  t0, x0, 0
```

```
add   t0, t0, t1
```

```
lw    t2, -28(x8)
```

```
sw    t2, 0(t0)
```

```
lw    t0, -8(x8)
```

```
sw    t0, -32(x8)
```

```
addi  t0, x0, 1
```

```
sw    t0, -36(x8)
```

```
lw    t0, -32(x8)
```

```
lw    t1, -36(x8)
```

```
add   t2, t0, t1
```

```
sw    t2, -40(x8)
```

```
lw    t0, -40(x8)
```

```
sw    t0, -8(x8)
```

```
jal x0, L6
```

L7:

```
addi t0, x0, 0
```

```
sw t0, -44(x8)
```

```
lw t0, -44(x8)
```

```
addi x2, x2, -4
```

```
sw t0, 0(x2)
```

```
addi t0, x0, 0
```

```
sw t0, -48(x8)
```

```
lw t0, -48(x8)
```

```
addi x2, x2, -4
```

```
sw t0, 0(x2)
```

```
addi t0, x0, 10
```

```
sw t0, -52(x8)
```

```
lw t0, -52(x8)
```

```
addi x2, x2, -4
sw    t0, 0(x2)
```

```
addi x2, x2, -4
sw    x8, 0(x2)
addi x2, x2, -4
sw    x1, 0(x2)
jal   x1, sort
lw    x1, 0(x2)
lw    x8, 4(x2)
addi x2, x2, 8
```

```
addi x2, x2, 12
sw    x10, -56(x8)
```

```
addi t0, x0, 0
```

```
sw    t0, -60(x8)
```

```
lw    t0, -60(x8)
```

```
sw    t0, -8(x8)
```

L8:

```
lw    t0, -8(x8)
```

```
sw    t0, -64(x8)
```

```
addi t0, x0, 10
```

```
sw    t0, -68(x8)
```

```
lw    t0, -64(x8)
```

```
lw    t1, -68(x8)
```

```
slt   t2, t0, t1
```

```
sw    t2, -72(x8)
```

```
lw    t0, -72(x8)
```

```
beq   t0, x0, L9
```

```
lw    t0, -8(x8)
```

```
sw    t0, -76(x8)
```

```
lw    t1, -76(x8)
```

```
addi t0, x0, 2
```

```
sll   t1, t1, t0
```

```
addi t0, x0, 0
```

```
add   t0, t0, t1
```

```
lw    t2, 0(t0)
```

```
sw    t2, -80(x8)
```

```
lw    t0, -80(x8)
```

```
addi x2, x2, -4
```

```
sw    t0, 0(x2)
```

```
lw    t0, 0(x2)
```

```
addi x2, x2, 4
```

```
sw    t0, 2040(x0)
```

```
lw    t0, -8(x8)
```

```
sw    t0, -84(x8)
```

```
addi t0, x0, 1
```

```
sw    t0, -88(x8)
```

```
lw    t0, -84(x8)
```

```
lw    t1, -88(x8)
```

```
add   t2, t0, t1
```

```
sw    t2, -92(x8)
```

```
lw    t0, -92(x8)
```

```
sw    t0, -8(x8)
```

```
jal x0, L8
```

L9:

```
# ---- fim de main ----
```

```
addi x2, x8, 0
```

```
jalr x0, x1, 0
```

A.2.2 Código Executável (hex) Completo

```
7ff00113
```

```
7ff10113
```

```
7ff10113
```

```
7ff10113
```

```
2fc000ef
```

```
0000006f
```

```
00010433
```

```
fa410113
```

```
00c42283
```

```
fe542e23
```

```
ffc42283
```

```
fe542c23
```

```
00c42283
```

```
fe542a23
```

```
ff442303
```

```
00200293
```

```
00531333
```

```
01042283
```

```
006282b3
```

```
0002a383
```

```
fe742823
```

```
ff042283
```

```
fe542623
```

```
00c42283
```

```
fe542423
```

```
00100293
```

```
fe542223
```

```
fe842283
```


fe442303
006283b3
fe742023
fe042283
fc542e23
fdc42283
fc542c23
00842283
fc542a23
fd842283
fd442303
0062a3b3
fc742823
fd042283
0a028863
fdc42283
fc542623
fcc42303
00200293
00531333
01042283
006282b3
0002a383
fc742423
fec42283
fc542223
fc842283
fc442303
0062a3b3
fc742023
fc042283
04028063
fdc42283
fa542e23
fbc42303
00200293
00531333
01042283
006282b3
0002a383
fa742c23

fb842283
fe542623
fdc42283
fa542a23
fb442283
fe542c23
fdc42283
fa542823
00100293
fa542623
fb042283
fac42303
006283b3
fa742423
fa842283
fc542e23
f31ff06f
ff842283
fa542223
fa442503
00040113
00008067
00040113
00008067
00010433
fa410113
00c42283
fe542e23
ffc42283
fe542c23
ff842283
fe542a23
00842283
fe542823
00100293
fe542623
ff042283
fec42303
406283b3
fe742423
ff442283

fe842303
0062a3b3
fe742223
fe442283
12028e63
01042283
fe542023
fe042283
ffc10113
00512023
ff842283
fc542e23
fdc42283
ffc10113
00512023
00842283
fc542c23
fd842283
ffc10113
00512023
ffc10113
00812023
ffc10113
00112023
e01ff0ef
00012083
00412403
00810113
00c10113
fca42a23
fd442283
fc542823
fd042283
fc542623
fcc42303
00200293
00531333
01042283
006282b3
0002a383
fc742423

fc842283
fc542223
fd042283
fc542023
ff842283
fa542e23
fbc42303
00200293
00531333
01042283
006282b3
0002a383
fa742c23
fc042303
00200293
00531333
01042283
006282b3
fb842383
0072a023
ff842283
fa542a23
fc442283
fa542823
fb442303
00200293
00531333
01042283
006282b3
fb042383
0072a023
ff842283
fa542623
00100293
fa542423
fac42283
fa842303
006283b3
fa742223
fa442283
fe542c23

e8dff06f
00040113
00008067
00010433
fa410113
00000293
fe542e23
ffc42283
fe542c23
ff842283
fe542a23
00a00293
fe542823
ff442283
ff042303
0062a3b3
fe742623
fec42283
04028e63
ff842283
fe542423
7fc02283
fe542223
fe842303
00200293
00531333
00000293
006282b3
fe442383
0072a023
ff842283
fe542023
00100293
fc542e23
fe042283
fdc42303
006283b3
fc742c23
fd842283
fe542c23
f85ff06f

00000293
fc542a23
fd442283
ffc10113
00512023
00000293
fc542823
fd042283
ffc10113
00512023
00a00293
fc542623
fcc42283
ffc10113
00512023
ffc10113
00812023
ffc10113
00112023
d85ff0ef
00012083
00412403
00810113
00c10113
fca42423
00000293
fc542223
fc442283
fe542c23
ff842283
fc542023
00a00293
fa542e23
fc042283
fbc42303
0062a3b3
fa742c23
fb842283
06028663
ff842283
fa542a23

```
fb442303
00200293
00531333
00000293
006282b3
0002a383
fa742823
fb042283
ffc10113
00512023
00012283
00410113
7e502c23
ff842283
fa542623
00100293
fa542423
fac42283
fa842303
006283b3
fa742223
fa442283
fe542c23
f75ff06f
00040113
00008067
```

A.3 Exemplo 3: Teste Completo

A.3.1 Código Intermediário Completo

```
1: (ALLOC, dados, 6, -)
2: (ALLOC, copia, 6, -)
3: (FUN, int, igual, -)
4: (ASSIGN, a, -, _t1)
5: (ASSIGN, b, -, _t2)
6: (EQ, _t1, _t2, _t3)
7: (IFF, _t3, L1, -)
8: (ASSIGN, 1, -, _t4)
9: (RET, _t4, -, -)
10: (GOTO, L2, -, -)
```

```

11: (LAB, L1, -, -)
12: (ASSIGN, 0, -, _t5)
13: (RET, _t5, -, -)
14: (LAB, L2, -, -)
15: (END, igual, -, -)
16: (FUN, int, diferente, -)
17: (ASSIGN, a, -, _t6)
18: (ASSIGN, b, -, _t7)
19: (NEQ, _t6, _t7, _t8)
20: (IFF, _t8, L3, -)
21: (ASSIGN, 1, -, _t9)
22: (RET, _t9, -, -)
23: (GOTO, L4, -, -)
24: (LAB, L3, -, -)
25: (ASSIGN, 0, -, _t10)
26: (RET, _t10, -, -)
27: (LAB, L4, -, -)
28: (END, diferente, -, -)
29: (FUN, int, maximo, -)
30: (ASSIGN, a, -, _t11)
31: (ASSIGN, b, -, _t12)
32: (GT, _t11, _t12, _t13)
33: (IFF, _t13, L5, -)
34: (ASSIGN, a, -, _t14)
35: (RET, _t14, -, -)
36: (GOTO, L6, -, -)
37: (LAB, L5, -, -)
38: (ASSIGN, b, -, _t15)
39: (RET, _t15, -, -)
40: (LAB, L6, -, -)
41: (END, maximo, -, -)
42: (FUN, int, distancia, -)
43: (ASSIGN, a, -, _t16)
44: (ASSIGN, b, -, _t17)
45: (GE, _t16, _t17, _t18)
46: (IFF, _t18, L7, -)
47: (ASSIGN, a, -, _t19)
48: (ASSIGN, b, -, _t20)
49: (SUB, _t19, _t20, _t21)
50: (RET, _t21, -, -)
51: (GOTO, L8, -, -)

```



```

52: (LAB, L7, -, -)
53: (ASSIGN, b, -, _t22)
54: (ASSIGN, a, -, _t23)
55: (SUB, _t22, _t23, _t24)
56: (RET, _t24, -, -)
57: (LAB, L8, -, -)
58: (END, distancia, -, -)
59: (FUN, int, fib, -)
60: (ASSIGN, n, -, _t25)
61: (ASSIGN, 1, -, _t26)
62: (LE, _t25, _t26, _t27)
63: (IFF, _t27, L9, -)
64: (ASSIGN, n, -, _t28)
65: (RET, _t28, -, -)
66: (GOTO, L10, -, -)
67: (LAB, L9, -, -)
68: (ASSIGN, n, -, _t29)
69: (ASSIGN, 1, -, _t30)
70: (SUB, _t29, _t30, _t31)
71: (PARAM, _t31, -, -)
72: (CALL, fib, 1, _t32)
73: (ASSIGN, n, -, _t33)
74: (ASSIGN, 2, -, _t34)
75: (SUB, _t33, _t34, _t35)
76: (PARAM, _t35, -, -)
77: (CALL, fib, 1, _t36)
78: (ADD, _t32, _t36, _t37)
79: (RET, _t37, -, -)
80: (LAB, L10, -, -)
81: (END, fib, -, -)
82: (FUN, int, resto, -)
83: (ASSIGN, a, -, _t38)
84: (ASSIGN, b, -, _t39)
85: (DIV, _t38, _t39, _t40)
86: (ASSIGN, _t40, -, q)
87: (ASSIGN, a, -, _t41)
88: (ASSIGN, q, -, _t42)
89: (ASSIGN, b, -, _t43)
90: (MULT, _t42, _t43, _t44)
91: (SUB, _t41, _t44, _t45)
92: (ASSIGN, _t45, -, r)

```

```

93: (ASSIGN, r, -, _t46)
94: (RET, _t46, -, -)
95: (END, resto, -, -)
96: (FUN, int, mdc, -)
97: (LAB, L11, -, -)
98: (ASSIGN, b, -, _t47)
99: (ASSIGN, 0, -, _t48)
100: (NEQ, _t47, _t48, _t49)
101: (IFF, _t49, L12, -)
102: (ASSIGN, a, -, _t50)
103: (PARAM, _t50, -, -)
104: (ASSIGN, b, -, _t51)
105: (PARAM, _t51, -, -)
106: (CALL, resto, 2, _t52)
107: (ASSIGN, _t52, -, r)
108: (ASSIGN, b, -, _t53)
109: (ASSIGN, _t53, -, a)
110: (ASSIGN, r, -, _t54)
111: (ASSIGN, _t54, -, b)
112: (GOTO, L11, -, -)
113: (LAB, L12, -, -)
114: (ASSIGN, a, -, _t55)
115: (RET, _t55, -, -)
116: (END, mdc, -, -)
117: (FUN, void, ler, -)
118: (ASSIGN, 0, -, _t56)
119: (ASSIGN, _t56, -, i)
120: (LAB, L13, -, -)
121: (ASSIGN, i, -, _t57)
122: (ASSIGN, n, -, _t58)
123: (LT, _t57, _t58, _t59)
124: (IFF, _t59, L14, -)
125: (ASSIGN, i, -, _t60)
126: (CALL, input, 0, _t61)
127: (STORE, _t61, _t60, v)
128: (ASSIGN, i, -, _t62)
129: (ASSIGN, 1, -, _t63)
130: (ADD, _t62, _t63, _t64)
131: (ASSIGN, _t64, -, i)
132: (GOTO, L13, -, -)
133: (LAB, L14, -, -)

```

```

134: (END, ler, -, -)
135: (FUN, void, copiareverso, -)
136: (ASSIGN, 0, -, _t65)
137: (ASSIGN, _t65, -, i)
138: (LAB, L15, -, -)
139: (ASSIGN, i, -, _t66)
140: (ASSIGN, n, -, _t67)
141: (LT, _t66, _t67, _t68)
142: (IFF, _t68, L16, -)
143: (ASSIGN, n, -, _t69)
144: (ASSIGN, 1, -, _t70)
145: (SUB, _t69, _t70, _t71)
146: (ASSIGN, i, -, _t72)
147: (SUB, _t71, _t72, _t73)
148: (ASSIGN, _t73, -, j)
149: (ASSIGN, i, -, _t74)
150: (ASSIGN, j, -, _t75)
151: (LOAD, origem, _t75, _t76)
152: (STORE, _t76, _t74, destino)
153: (ASSIGN, i, -, _t77)
154: (ASSIGN, 1, -, _t78)
155: (ADD, _t77, _t78, _t79)
156: (ASSIGN, _t79, -, i)
157: (GOTO, L15, -, -)
158: (LAB, L16, -, -)
159: (END, copiareverso, -, -)
160: (FUN, int, soma, -)
161: (ASSIGN, 0, -, _t80)
162: (ASSIGN, _t80, -, i)
163: (ASSIGN, 0, -, _t81)
164: (ASSIGN, _t81, -, s)
165: (LAB, L17, -, -)
166: (ASSIGN, i, -, _t82)
167: (ASSIGN, n, -, _t83)
168: (LT, _t82, _t83, _t84)
169: (IFF, _t84, L18, -)
170: (ASSIGN, s, -, _t85)
171: (ASSIGN, i, -, _t86)
172: (LOAD, v, _t86, _t87)
173: (ADD, _t85, _t87, _t88)
174: (ASSIGN, _t88, -, s)

```

```

175: (ASSIGN, i, -, _t89)
176: (ASSIGN, 1, -, _t90)
177: (ADD, _t89, _t90, _t91)
178: (ASSIGN, _t91, -, i)
179: (GOTO, L17, -, -)
180: (LAB, L18, -, -)
181: (ASSIGN, s, -, _t92)
182: (RET, _t92, -, -)
183: (END, soma, -, -)
184: (FUN, int, produto, -)
185: (ASSIGN, 0, -, _t93)
186: (ASSIGN, _t93, -, i)
187: (ASSIGN, 1, -, _t94)
188: (ASSIGN, _t94, -, p)
189: (LAB, L19, -, -)
190: (ASSIGN, i, -, _t95)
191: (ASSIGN, n, -, _t96)
192: (LT, _t95, _t96, _t97)
193: (IFF, _t97, L20, -)
194: (ASSIGN, i, -, _t98)
195: (LOAD, v, _t98, _t99)
196: (ASSIGN, 0, -, _t100)
197: (GT, _t99, _t100, _t101)
198: (IFF, _t101, L21, -)
199: (ASSIGN, p, -, _t102)
200: (ASSIGN, i, -, _t103)
201: (LOAD, v, _t103, _t104)
202: (MULT, _t102, _t104, _t105)
203: (ASSIGN, _t105, -, p)
204: (GOTO, L22, -, -)
205: (LAB, L21, -, -)
206: (ASSIGN, p, -, _t106)
207: (ASSIGN, 1, -, _t107)
208: (MULT, _t106, _t107, _t108)
209: (ASSIGN, _t108, -, p)
210: (LAB, L22, -, -)
211: (ASSIGN, i, -, _t109)
212: (ASSIGN, 1, -, _t110)
213: (ADD, _t109, _t110, _t111)
214: (ASSIGN, _t111, -, i)
215: (GOTO, L19, -, -)

```

```
216: (LAB, L20, -, -)
217: (ASSIGN, p, -, _t112)
218: (RET, _t112, -, -)
219: (END, produto, -, -)
220: (FUN, void, main, -)
221: (ASSIGN, dados, -, _t113)
222: (PARAM, _t113, -, -)
223: (ASSIGN, 6, -, _t114)
224: (PARAM, _t114, -, -)
225: (CALL, ler, 2, _t115)
226: (ASSIGN, dados, -, _t116)
227: (PARAM, _t116, -, -)
228: (ASSIGN, copia, -, _t117)
229: (PARAM, _t117, -, -)
230: (ASSIGN, 6, -, _t118)
231: (PARAM, _t118, -, -)
232: (CALL, copiareverso, 3, _t119)
233: (ASSIGN, dados, -, _t120)
234: (PARAM, _t120, -, -)
235: (ASSIGN, 6, -, _t121)
236: (PARAM, _t121, -, -)
237: (CALL, soma, 2, _t122)
238: (ASSIGN, _t122, -, s)
239: (ASSIGN, dados, -, _t123)
240: (PARAM, _t123, -, -)
241: (ASSIGN, 6, -, _t124)
242: (PARAM, _t124, -, -)
243: (CALL, produto, 2, _t125)
244: (ASSIGN, _t125, -, p)
245: (ASSIGN, 0, -, _t126)
246: (LOAD, dados, _t126, _t127)
247: (PARAM, _t127, -, -)
248: (ASSIGN, 5, -, _t128)
249: (LOAD, dados, _t128, _t129)
250: (PARAM, _t129, -, -)
251: (CALL, maximo, 2, _t130)
252: (ASSIGN, _t130, -, m)
253: (ASSIGN, 1, -, _t131)
254: (LOAD, dados, _t131, _t132)
255: (PARAM, _t132, -, -)
256: (ASSIGN, 4, -, _t133)
```

257: (LOAD, dados, _t133, _t134)
258: (PARAM, _t134, -, -)
259: (CALL, distancia, 2, _t135)
260: (ASSIGN, _t135, -, d)
261: (ASSIGN, 6, -, _t136)
262: (PARAM, _t136, -, -)
263: (CALL, fib, 1, _t137)
264: (ASSIGN, _t137, -, f)
265: (ASSIGN, 2, -, _t138)
266: (LOAD, dados, _t138, _t139)
267: (PARAM, _t139, -, -)
268: (ASSIGN, 3, -, _t140)
269: (LOAD, dados, _t140, _t141)
270: (PARAM, _t141, -, -)
271: (CALL, igual, 2, _t142)
272: (ASSIGN, _t142, -, eq)
273: (ASSIGN, 2, -, _t143)
274: (LOAD, dados, _t143, _t144)
275: (PARAM, _t144, -, -)
276: (ASSIGN, 3, -, _t145)
277: (LOAD, dados, _t145, _t146)
278: (PARAM, _t146, -, -)
279: (CALL, diferente, 2, _t147)
280: (ASSIGN, _t147, -, ne)
281: (ASSIGN, s, -, _t148)
282: (PARAM, _t148, -, -)
283: (ASSIGN, 5, -, _t149)
284: (PARAM, _t149, -, -)
285: (CALL, resto, 2, _t150)
286: (ASSIGN, _t150, -, r)
287: (ASSIGN, s, -, _t151)
288: (PARAM, _t151, -, -)
289: (ASSIGN, p, -, _t152)
290: (PARAM, _t152, -, -)
291: (CALL, mdc, 2, _t153)
292: (ASSIGN, _t153, -, g)
293: (ASSIGN, 0, -, _t154)
294: (LOAD, copia, _t154, _t155)
295: (ASSIGN, _t155, -, primeiro)
296: (ASSIGN, 5, -, _t156)
297: (LOAD, copia, _t156, _t157)

```

298: (ASSIGN, _t157, -, ultimo)
299: (ASSIGN, s, -, _t158)
300: (PARAM, _t158, -, -)
301: (CALL, output, 1, -)
302: (ASSIGN, p, -, _t159)
303: (PARAM, _t159, -, -)
304: (CALL, output, 1, -)
305: (ASSIGN, m, -, _t160)
306: (PARAM, _t160, -, -)
307: (CALL, output, 1, -)
308: (ASSIGN, d, -, _t161)
309: (PARAM, _t161, -, -)
310: (CALL, output, 1, -)
311: (ASSIGN, f, -, _t162)
312: (PARAM, _t162, -, -)
313: (CALL, output, 1, -)
314: (ASSIGN, eq, -, _t163)
315: (PARAM, _t163, -, -)
316: (CALL, output, 1, -)
317: (ASSIGN, ne, -, _t164)
318: (PARAM, _t164, -, -)
319: (CALL, output, 1, -)
320: (ASSIGN, r, -, _t165)
321: (PARAM, _t165, -, -)
322: (CALL, output, 1, -)
323: (ASSIGN, g, -, _t166)
324: (PARAM, _t166, -, -)
325: (CALL, output, 1, -)
326: (ASSIGN, primeiro, -, _t167)
327: (PARAM, _t167, -, -)
328: (CALL, output, 1, -)
329: (ASSIGN, ultimo, -, _t168)
330: (PARAM, _t168, -, -)
331: (CALL, output, 1, -)
332: (END, main, -, -)

```

A.3.2 Código Assembly Completo

```

# Assembly RISC-V gerado pelo Compilador C-
# Arquitetura: RV32IM customizada
# I via MMIO: endereco 2044

```

```

# 0 via MMIO: endereco 2040

    addi x2, x0, 2047
    addi x2, x2, 2047
    addi x2, x2, 2047
    addi x2, x2, 2047
    jal  x1, main
__halt:
    jal  x0, __halt

# vetor global 'dados[6]' reservado no endereco 0
# vetor global 'copia[6]' reservado no endereco 24


# ---- funcao igual (int) | 2 parametros ----
igual:
    add  x8, x2, x0
    addi x2, x2, -20

    lw   t0, 12(x8)

    sw   t0, -4(x8)

    lw   t0, 8(x8)

    sw   t0, -8(x8)

    lw   t0, -4(x8)

    lw   t1, -8(x8)

```



```

xor   t2, t0, t1
slti  t2, t2, 1

sw    t2, -12(x8)

lw    t0, -12(x8)

beq   t0, x0, L1

addi  t0, x0, 1

sw    t0, -16(x8)

lw    x10, -16(x8)

addi  x2, x8, 0
jalr  x0, x1, 0

jal   x0, L2

```

L1:

```

addi  t0, x0, 0

sw    t0, -20(x8)

lw    x10, -20(x8)

```

```
addi x2, x8, 0
jalr x0, x1, 0
```

L2:

---- fim de igual ----

```
addi x2, x8, 0
jalr x0, x1, 0
```

---- funcao diferente (int) | 2 parametros ----

diferente:

```
add  x8, x2, x0
addi x2, x2, -20
```

```
lw    t0, 12(x8)
```

```
sw    t0, -4(x8)
```

```
lw    t0, 8(x8)
```

```
sw    t0, -8(x8)
```

```
lw    t0, -4(x8)
```

```
lw    t1, -8(x8)
```

```
xor  t2, t0, t1
slti t2, t2, 1
xori t2, t2, 1
```

```
sw   t2, -12(x8)
```

```
lw   t0, -12(x8)
```

```
beq  t0, x0, L3
```

```
addi t0, x0, 1
```

```
sw   t0, -16(x8)
```

```
lw   x10, -16(x8)
```

```
addi x2, x8, 0
jalr x0, x1, 0
```

```
jal  x0, L4
```

L3:

```
addi t0, x0, 0
```

```
sw   t0, -20(x8)
```

```
lw   x10, -20(x8)
```

```
    addi x2, x8, 0
    jalr x0, x1, 0
```

L4:

```
# ---- fim de diferente ----
    addi x2, x8, 0
    jalr x0, x1, 0
```

```
# ---- funcao maximo (int) | 2 parametros ----
maximo:
```

```
    add  x8, x2, x0
    addi x2, x2, -20
```

```
    lw   t0, 12(x8)
```

```
    sw   t0, -4(x8)
```

```
    lw   t0, 8(x8)
```

```
    sw   t0, -8(x8)
```

```
    lw   t0, -4(x8)
```

```
lw    t1, -8(x8)
```

```
slt   t2, t1, t0
```

```
sw    t2, -12(x8)
```

```
lw    t0, -12(x8)
```

```
beq   t0, x0, L5
```

```
lw    t0, 12(x8)
```

```
sw    t0, -16(x8)
```

```
lw    x10, -16(x8)
```

```
addi  x2, x8, 0
```

```
jalr  x0, x1, 0
```

```
jal   x0, L6
```

L5:

```
lw    t0, 8(x8)
```

```
sw    t0, -20(x8)
```

```
lw    x10, -20(x8)
```

```
    addi x2, x8, 0
    jalr x0, x1, 0
```

L6:

```
# ---- fim de maximo ----
```

```
    addi x2, x8, 0
    jalr x0, x1, 0
```

```
# ---- funcao distancia (int) | 2 parametros ----
```

distancia:

```
    add  x8, x2, x0
    addi x2, x2, -36
```

```
    lw   t0, 12(x8)
```

```
    sw   t0, -4(x8)
```

```
    lw   t0, 8(x8)
```

```
    sw   t0, -8(x8)
```

```
    lw   t0, -4(x8)
```

```
lw    t1, -8(x8)

slt   t2, t0, t1
xori  t2, t2, 1

sw    t2, -12(x8)

lw    t0, -12(x8)

beq   t0, x0, L7

lw    t0, 12(x8)

sw    t0, -16(x8)

lw    t0, 8(x8)

sw    t0, -20(x8)

lw    t0, -16(x8)

lw    t1, -20(x8)

sub   t2, t0, t1

sw    t2, -24(x8)

lw    x10, -24(x8)
```

```
addi x2, x8, 0
jalr x0, x1, 0
```

```
jal x0, L8
```

L7:

```
lw t0, 8(x8)
```

```
sw t0, -28(x8)
```

```
lw t0, 12(x8)
```

```
sw t0, -32(x8)
```

```
lw t0, -28(x8)
```

```
lw t1, -32(x8)
```

```
sub t2, t0, t1
```

```
sw t2, -36(x8)
```

```
lw x10, -36(x8)
```

```
addi x2, x8, 0
jalr x0, x1, 0
```


L8:

---- fim de distancia ----

addi x2, x8, 0

jalr x0, x1, 0

---- funcao fib (int) | 1 parametros ----

fib:

add x8, x2, x0

addi x2, x2, -52

lw t0, 8(x8)

sw t0, -4(x8)

addi t0, x0, 1

sw t0, -8(x8)

lw t0, -4(x8)

lw t1, -8(x8)

slt t2, t1, t0

xori t2, t2, 1

```
sw    t2, -12(x8)
```

```
lw     t0, -12(x8)
```

```
beq    t0, x0, L9
```

```
lw     t0, 8(x8)
```

```
sw     t0, -16(x8)
```

```
lw     x10, -16(x8)
```

```
addi   x2, x8, 0
```

```
jalr   x0, x1, 0
```

```
jal    x0, L10
```

L9:

```
lw     t0, 8(x8)
```

```
sw     t0, -20(x8)
```

```
addi   t0, x0, 1
```

```
sw     t0, -24(x8)
```

```
lw    t0, -20(x8)
```

```
lw    t1, -24(x8)
```

```
sub   t2, t0, t1
```

```
sw    t2, -28(x8)
```

```
lw    t0, -28(x8)
```

```
addi  x2, x2, -4
```

```
sw    t0, 0(x2)
```

```
addi  x2, x2, -4
```

```
sw    x8, 0(x2)
```

```
addi  x2, x2, -4
```

```
sw    x1, 0(x2)
```

```
jal   x1, fib
```

```
lw    x1, 0(x2)
```

```
lw    x8, 4(x2)
```

```
addi  x2, x2, 8
```

```
addi  x2, x2, 4
```

```
sw    x10, -32(x8)
```

```
lw    t0, 8(x8)
```

```
sw    t0, -36(x8)
```

```

addi t0, x0, 2

sw    t0, -40(x8)

lw    t0, -36(x8)

lw    t1, -40(x8)

sub   t2, t0, t1

sw    t2, -44(x8)

lw    t0, -44(x8)

addi x2, x2, -4
sw    t0, 0(x2)

addi x2, x2, -4
sw    x8, 0(x2)
addi x2, x2, -4
sw    x1, 0(x2)
jal   x1, fib
lw    x1, 0(x2)
lw    x8, 4(x2)
addi x2, x2, 8

addi x2, x2, 4
sw    x10, -48(x8)

```

```
lw    t0, -32(x8)
```

```
lw    t1, -48(x8)
```

```
add   t2, t0, t1
```

```
sw    t2, -52(x8)
```

```
lw    x10, -52(x8)
```

```
addi  x2, x8, 0
```

```
jalr  x0, x1, 0
```

L10:

```
# ---- fim de fib ----
```

```
addi  x2, x8, 0
```

```
jalr  x0, x1, 0
```

```
# ---- funcao resto (int) | 2 parametros ----
```

```
resto:
```

```
add   x8, x2, x0
```

```
addi  x2, x2, -44
```

```
lw    t0, 12(x8)
```

```
sw    t0, -4(x8)
```

```
lw    t0, 8(x8)
```

```
sw    t0, -8(x8)
```

```
lw    t0, -4(x8)
```

```
lw    t1, -8(x8)
```

```
div   t2, t0, t1
```

```
sw    t2, -12(x8)
```

```
lw    t0, -12(x8)
```

```
sw    t0, -16(x8)
```

```
lw    t0, 12(x8)
```

```
sw    t0, -20(x8)
```

```
lw    t0, -16(x8)
```

```
sw    t0, -24(x8)
```

```
lw    t0, 8(x8)
```

```
sw    t0, -28(x8)
```

```
lw    t0, -24(x8)
```

```
lw    t1, -28(x8)
```

```
mul   t2, t0, t1
```

```
sw    t2, -32(x8)
```

```
lw    t0, -20(x8)
```

```
lw    t1, -32(x8)
```

```
sub   t2, t0, t1
```

```
sw    t2, -36(x8)
```

```
lw    t0, -36(x8)
```

```
sw    t0, -40(x8)
```

```
lw    t0, -40(x8)
```

```
sw    t0, -44(x8)
```

```

    lw    x10, -44(x8)

    addi x2, x8, 0
    jalr x0, x1, 0

# ---- fim de resto ----
    addi x2, x8, 0
    jalr x0, x1, 0

# ---- funcao mdc (int) | 2 parametros ----
mdc:
    add  x8, x2, x0
    addi x2, x2, -40

L11:

    lw    t0, 8(x8)

    sw    t0, -4(x8)

    addi t0, x0, 0

    sw    t0, -8(x8)

    lw    t0, -4(x8)

```



```

lw    t1, -8(x8)

xor    t2, t0, t1
slti   t2, t2, 1
xori   t2, t2, 1

sw     t2, -12(x8)

lw     t0, -12(x8)

beq    t0, x0, L12

lw     t0, 12(x8)

sw     t0, -16(x8)

lw     t0, -16(x8)

addi   x2, x2, -4
sw     t0, 0(x2)

lw     t0, 8(x8)

sw     t0, -20(x8)

lw     t0, -20(x8)

addi   x2, x2, -4

```

```
sw    t0, 0(x2)
```

```
addi x2, x2, -4
```

```
sw    x8, 0(x2)
```

```
addi x2, x2, -4
```

```
sw    x1, 0(x2)
```

```
jal   x1, resto
```

```
lw    x1, 0(x2)
```

```
lw    x8, 4(x2)
```

```
addi x2, x2, 8
```

```
addi x2, x2, 8
```

```
sw    x10, -24(x8)
```

```
lw    t0, -24(x8)
```

```
sw    t0, -28(x8)
```

```
lw    t0, 8(x8)
```

```
sw    t0, -32(x8)
```

```
lw    t0, -32(x8)
```

```
sw    t0, 12(x8)
```

```
lw    t0, -28(x8)
```

```
sw    t0, -36(x8)
```

```

    lw    t0, -36(x8)

    sw    t0, 8(x8)

    jal   x0, L11

L12:

    lw    t0, 12(x8)

    sw    t0, -40(x8)

    lw    x10, -40(x8)

    addi  x2, x8, 0
    jalr  x0, x1, 0

# ---- fim de mdc ----
    addi  x2, x8, 0
    jalr  x0, x1, 0

# ---- funcao ler (void) | 2 parametros ----
ler:
    add   x8, x2, x0

```

```
addi x2, x2, -40
```

```
addi t0, x0, 0
```

```
sw    t0, -4(x8)
```

```
lw    t0, -4(x8)
```

```
sw    t0, -8(x8)
```

L13:

```
lw    t0, -8(x8)
```

```
sw    t0, -12(x8)
```

```
lw    t0, 8(x8)
```

```
sw    t0, -16(x8)
```

```
lw    t0, -12(x8)
```

```
lw    t1, -16(x8)
```

```
slt   t2, t0, t1
```

```
sw    t2, -20(x8)
```

```
lw    t0, -20(x8)
```

```
beq    t0, x0, L14
```

```
lw    t0, -8(x8)
```

```
sw    t0, -24(x8)
```

```
lw    t0, 2044(x0)
```

```
sw    t0, -28(x8)
```

```
lw    t1, -24(x8)
```

```
addi   t0, x0, 2
```

```
sll    t1, t1, t0
```

```
lw    t0, 12(x8)
```

```
add    t0, t0, t1
```

```
lw    t2, -28(x8)
```

```
sw    t2, 0(t0)
```

```
lw    t0, -8(x8)
```

```
sw    t0, -32(x8)
```

```
addi   t0, x0, 1
```

```
sw    t0, -36(x8)
```

```
lw    t0, -32(x8)
```

```
lw    t1, -36(x8)
```

```
add   t2, t0, t1
```

```
sw    t2, -40(x8)
```

```
lw    t0, -40(x8)
```

```
sw    t0, -8(x8)
```

```
jal   x0, L13
```

L14:

```
# ---- fim de ler ----
```

```
addi  x2, x8, 0
```

```
jalr  x0, x1, 0
```

```
# ---- funcao copiareverso (void) | 3 parametros ----
```

```
copiareverso:
```

```
add   x8, x2, x0
```

```
addi x2, x2, -68
```

```
addi t0, x0, 0
```

```
sw    t0, -4(x8)
```

```
lw    t0, -4(x8)
```

```
sw    t0, -8(x8)
```

L15:

```
lw    t0, -8(x8)
```

```
sw    t0, -12(x8)
```

```
lw    t0, 8(x8)
```

```
sw    t0, -16(x8)
```

```
lw    t0, -12(x8)
```

```
lw    t1, -16(x8)
```

```
slt   t2, t0, t1
```

```
sw    t2, -20(x8)
```

```
lw    t0, -20(x8)
```

```
beq   t0, x0, L16
```

```
lw    t0, 8(x8)
```

```
sw    t0, -24(x8)
```

```
addi  t0, x0, 1
```

```
sw    t0, -28(x8)
```

```
lw    t0, -24(x8)
```

```
lw    t1, -28(x8)
```

```
sub   t2, t0, t1
```

```
sw    t2, -32(x8)
```

```
lw    t0, -8(x8)
```

```
sw    t0, -36(x8)
```

```
lw    t0, -32(x8)
```

```
lw    t1, -36(x8)
```



```

sub    t2, t0, t1

sw     t2, -40(x8)

lw     t0, -40(x8)

sw     t0, -44(x8)

lw     t0, -8(x8)

sw     t0, -48(x8)

lw     t0, -44(x8)

sw     t0, -52(x8)

lw     t1, -52(x8)


addi   t0, x0, 2
sll    t1, t1, t0
lw     t0, 16(x8)
add    t0, t0, t1
lw     t2, 0(t0)
sw     t2, -56(x8)


lw     t1, -48(x8)


addi   t0, x0, 2

```

```
sll  t1, t1, t0
lw   t0, 12(x8)
add  t0, t0, t1
lw   t2, -56(x8)
```

```
sw   t2, 0(t0)
lw   t0, -8(x8)
```

```
sw   t0, -60(x8)
```

```
addi t0, x0, 1
```

```
sw   t0, -64(x8)
```

```
lw   t0, -60(x8)
```

```
lw   t1, -64(x8)
```

```
add  t2, t0, t1
```

```
sw   t2, -68(x8)
```

```
lw   t0, -68(x8)
```

```
sw   t0, -8(x8)
```

```
jal  x0, L15
```

L16:

```

# ---- fim de copiareverso ----
    addi x2, x8, 0
    jalr x0, x1, 0


# ---- funcao soma (int) | 2 parametros ----
soma:
    add  x8, x2, x0
    addi x2, x2, -60


    addi t0, x0, 0


    sw    t0, -4(x8)


    lw    t0, -4(x8)


    sw    t0, -8(x8)


    addi t0, x0, 0


    sw    t0, -12(x8)


    lw    t0, -12(x8)


    sw    t0, -16(x8)

```

L17:

lw t0, -8(x8)

sw t0, -20(x8)

lw t0, 8(x8)

sw t0, -24(x8)

lw t0, -20(x8)

lw t1, -24(x8)

slt t2, t0, t1

sw t2, -28(x8)

lw t0, -28(x8)

beq t0, x0, L18

lw t0, -16(x8)

sw t0, -32(x8)

```
lw    t0, -8(x8)

sw    t0, -36(x8)

lw    t1, -36(x8)

addi  t0, x0, 2
sll   t1, t1, t0
lw    t0, 12(x8)
add   t0, t0, t1
lw    t2, 0(t0)
sw    t2, -40(x8)

lw    t0, -32(x8)

lw    t1, -40(x8)

add   t2, t0, t1

sw    t2, -44(x8)

lw    t0, -44(x8)

sw    t0, -16(x8)

lw    t0, -8(x8)

sw    t0, -48(x8)
```

```
addi t0, x0, 1
```

```
sw    t0, -52(x8)
```

```
lw    t0, -48(x8)
```

```
lw    t1, -52(x8)
```

```
add   t2, t0, t1
```

```
sw    t2, -56(x8)
```

```
lw    t0, -56(x8)
```

```
sw    t0, -8(x8)
```

```
jal   x0, L17
```

L18:

```
lw    t0, -16(x8)
```

```
sw    t0, -60(x8)
```

```
lw    x10, -60(x8)
```

```
addi x2, x8, 0
```

```
jalr x0, x1, 0
```

```
# ---- fim de soma ----
```

```
    addi x2, x8, 0
```

```
    jalr x0, x1, 0
```

```
# ---- funcao produto (int) | 2 parametros ----
```

```
produto:
```

```
    add  x8, x2, x0
```

```
    addi x2, x2, -88
```

```
    addi t0, x0, 0
```

```
    sw   t0, -4(x8)
```

```
    lw   t0, -4(x8)
```

```
    sw   t0, -8(x8)
```

```
    addi t0, x0, 1
```

```
    sw   t0, -12(x8)
```

```
    lw   t0, -12(x8)
```

```
    sw   t0, -16(x8)
```

L19:

lw t0, -8(x8)

sw t0, -20(x8)

lw t0, 8(x8)

sw t0, -24(x8)

lw t0, -20(x8)

lw t1, -24(x8)

slt t2, t0, t1

sw t2, -28(x8)

lw t0, -28(x8)

beq t0, x0, L20

lw t0, -8(x8)

sw t0, -32(x8)


```
lw    t1, -32(x8)
```

```
addi  t0, x0, 2
```

```
sll   t1, t1, t0
```

```
lw    t0, 12(x8)
```

```
add   t0, t0, t1
```

```
lw    t2, 0(t0)
```

```
sw    t2, -36(x8)
```

```
addi  t0, x0, 0
```

```
sw    t0, -40(x8)
```

```
lw    t0, -36(x8)
```

```
lw    t1, -40(x8)
```

```
slt   t2, t1, t0
```

```
sw    t2, -44(x8)
```

```
lw    t0, -44(x8)
```

```
beq   t0, x0, L21
```

```
lw    t0, -16(x8)
```

```
sw    t0, -48(x8)
```

```

lw    t0, -8(x8)

sw    t0, -52(x8)

lw    t1, -52(x8)

addi  t0, x0, 2
sll   t1, t1, t0
lw    t0, 12(x8)
add   t0, t0, t1
lw    t2, 0(t0)
sw    t2, -56(x8)

lw    t0, -48(x8)

lw    t1, -56(x8)

mul   t2, t0, t1

sw    t2, -60(x8)

lw    t0, -60(x8)

sw    t0, -16(x8)

jal   x0, L22

```

L21:

```
lw    t0, -16(x8)
```

```
sw    t0, -64(x8)
```

```
addi  t0, x0, 1
```

```
sw    t0, -68(x8)
```

```
lw    t0, -64(x8)
```

```
lw    t1, -68(x8)
```

```
mul   t2, t0, t1
```

```
sw    t2, -72(x8)
```

```
lw    t0, -72(x8)
```

```
sw    t0, -16(x8)
```

L22:

```
lw    t0, -8(x8)
```

```
sw    t0, -76(x8)
```

```
addi  t0, x0, 1
```

```
sw    t0, -80(x8)
```

```
lw    t0, -76(x8)
```

```
lw    t1, -80(x8)
```

```
add   t2, t0, t1
```

```
sw    t2, -84(x8)
```

```
lw    t0, -84(x8)
```

```
sw    t0, -8(x8)
```

```
jal   x0, L19
```

L20:

```
lw    t0, -16(x8)
```

```
sw    t0, -88(x8)
```

```
lw    x10, -88(x8)
```

```
addi  x2, x8, 0
```

```
jalr  x0, x1, 0
```

```

# ---- fim de produto ----
    addi x2, x8, 0
    jalr x0, x1, 0


# ---- funcao main (void) | 0 parametros ----
main:
    add  x8, x2, x0
    addi x2, x2, -268


    addi t0, x0, 0


    sw    t0, -4(x8)


    lw    t0, -4(x8)


    addi x2, x2, -4
    sw    t0, 0(x2)


    addi t0, x0, 6


    sw    t0, -8(x8)


    lw    t0, -8(x8)


    addi x2, x2, -4
    sw    t0, 0(x2)


    addi x2, x2, -4
    sw    x8, 0(x2)

```

```
addi x2, x2, -4
sw    x1, 0(x2)
jal   x1, ler
lw    x1, 0(x2)
lw    x8, 4(x2)
addi  x2, x2, 8
```

```
addi x2, x2, 8
sw    x10, -12(x8)
```

```
addi t0, x0, 0
```

```
sw    t0, -16(x8)
```

```
lw    t0, -16(x8)
```

```
addi x2, x2, -4
sw    t0, 0(x2)
```

```
addi t0, x0, 24
```

```
sw    t0, -20(x8)
```

```
lw    t0, -20(x8)
```

```
addi x2, x2, -4
sw    t0, 0(x2)
```

```
addi t0, x0, 6
```

```
sw    t0, -24(x8)
```

```
lw    t0, -24(x8)
```

```
addi x2, x2, -4  
sw    t0, 0(x2)
```

```
addi x2, x2, -4  
sw    x8, 0(x2)  
addi x2, x2, -4  
sw    x1, 0(x2)  
jal   x1, copiareverso  
lw    x1, 0(x2)  
lw    x8, 4(x2)  
addi x2, x2, 8
```

```
addi x2, x2, 12  
sw    x10, -28(x8)
```

```
addi t0, x0, 0
```

```
sw    t0, -32(x8)
```

```
lw    t0, -32(x8)
```

```
addi x2, x2, -4  
sw    t0, 0(x2)
```

```
addi t0, x0, 6
```

```
sw    t0, -36(x8)
```

```
lw    t0, -36(x8)
```

```
addi x2, x2, -4
```

```
sw    t0, 0(x2)
```

```
addi x2, x2, -4
```

```
sw    x8, 0(x2)
```

```
addi x2, x2, -4
```

```
sw    x1, 0(x2)
```

```
jal   x1, soma
```

```
lw    x1, 0(x2)
```

```
lw    x8, 4(x2)
```

```
addi x2, x2, 8
```

```
addi x2, x2, 8
```

```
sw    x10, -40(x8)
```

```
lw    t0, -40(x8)
```

```
sw    t0, -44(x8)
```

```
addi t0, x0, 0
```

```
sw    t0, -48(x8)
```



```
lw    t0, -48(x8)
```

```
addi x2, x2, -4
```

```
sw    t0, 0(x2)
```

```
addi t0, x0, 6
```

```
sw    t0, -52(x8)
```

```
lw    t0, -52(x8)
```

```
addi x2, x2, -4
```

```
sw    t0, 0(x2)
```

```
addi x2, x2, -4
```

```
sw    x8, 0(x2)
```

```
addi x2, x2, -4
```

```
sw    x1, 0(x2)
```

```
jal   x1, produto
```

```
lw    x1, 0(x2)
```

```
lw    x8, 4(x2)
```

```
addi x2, x2, 8
```

```
addi x2, x2, 8
```

```
sw    x10, -56(x8)
```

```
lw    t0, -56(x8)
```

```
sw    t0, -60(x8)
```

```
addi t0, x0, 0
```

```
sw    t0, -64(x8)
```

```
lw    t1, -64(x8)
```

```
addi t0, x0, 2
```

```
sll  t1, t1, t0
```

```
addi t0, x0, 0
```

```
add  t0, t0, t1
```

```
lw    t2, 0(t0)
```

```
sw    t2, -68(x8)
```

```
lw    t0, -68(x8)
```

```
addi x2, x2, -4
```

```
sw    t0, 0(x2)
```

```
addi t0, x0, 5
```

```
sw    t0, -72(x8)
```

```
lw    t1, -72(x8)
```

```
addi t0, x0, 2
```

```
sll  t1, t1, t0
```

```
addi t0, x0, 0
```

```
add  t0, t0, t1
```

```
lw    t2, 0(t0)
```

```
sw    t2, -76(x8)
```

```
lw    t0, -76(x8)
```

```
addi x2, x2, -4  
sw    t0, 0(x2)
```

```
addi x2, x2, -4  
sw    x8, 0(x2)  
addi x2, x2, -4  
sw    x1, 0(x2)  
jal   x1, maximo  
lw    x1, 0(x2)  
lw    x8, 4(x2)  
addi x2, x2, 8
```

```
addi x2, x2, 8  
sw    x10, -80(x8)
```

```
lw    t0, -80(x8)
```

```
sw    t0, -84(x8)
```

```
addi t0, x0, 1
```

```
sw    t0, -88(x8)
```

```
lw    t1, -88(x8)
```

```
addi t0, x0, 2  
sll  t1, t1, t0
```

```

addi t0, x0, 0
add  t0, t0, t1
lw   t2, 0(t0)
sw   t2, -92(x8)

lw   t0, -92(x8)

addi x2, x2, -4
sw   t0, 0(x2)

addi t0, x0, 4

sw   t0, -96(x8)

lw   t1, -96(x8)

addi t0, x0, 2
sll  t1, t1, t0
addi t0, x0, 0
add  t0, t0, t1
lw   t2, 0(t0)
sw   t2, -100(x8)

lw   t0, -100(x8)

addi x2, x2, -4
sw   t0, 0(x2)

addi x2, x2, -4
sw   x8, 0(x2)
addi x2, x2, -4
sw   x1, 0(x2)

```

```
jal x1, distancia
lw x1, 0(x2)
lw x8, 4(x2)
addi x2, x2, 8
```

```
addi x2, x2, 8
sw x10, -104(x8)
```

```
lw t0, -104(x8)
```

```
sw t0, -108(x8)
```

```
addi t0, x0, 6
```

```
sw t0, -112(x8)
```

```
lw t0, -112(x8)
```

```
addi x2, x2, -4
sw t0, 0(x2)
```

```
addi x2, x2, -4
sw x8, 0(x2)
addi x2, x2, -4
sw x1, 0(x2)
jal x1, fib
lw x1, 0(x2)
lw x8, 4(x2)
addi x2, x2, 8
```

```
addi x2, x2, 4
sw    x10, -116(x8)
```

```
lw    t0, -116(x8)
```

```
sw    t0, -120(x8)
```

```
addi t0, x0, 2
```

```
sw    t0, -124(x8)
```

```
lw    t1, -124(x8)
```

```
addi t0, x0, 2
sll  t1, t1, t0
addi t0, x0, 0
add  t0, t0, t1
lw   t2, 0(t0)
sw   t2, -128(x8)
```

```
lw    t0, -128(x8)
```

```
addi x2, x2, -4
sw    t0, 0(x2)
```

```
addi t0, x0, 3
```

```
sw    t0, -132(x8)
```

```
lw    t1, -132(x8)
```

```
addi  t0, x0, 2  
sll   t1, t1, t0  
addi  t0, x0, 0  
add   t0, t0, t1  
lw    t2, 0(t0)  
sw    t2, -136(x8)
```

```
lw    t0, -136(x8)
```

```
addi  x2, x2, -4  
sw    t0, 0(x2)
```

```
addi  x2, x2, -4  
sw    x8, 0(x2)  
addi  x2, x2, -4  
sw    x1, 0(x2)  
jal   x1, igual  
lw    x1, 0(x2)  
lw    x8, 4(x2)  
addi  x2, x2, 8
```

```
addi  x2, x2, 8  
sw    x10, -140(x8)
```

```
lw    t0, -140(x8)
```

```
sw    t0, -144(x8)
```

```

addi t0, x0, 2

sw    t0, -148(x8)

lw    t1, -148(x8)


addi t0, x0, 2
sll  t1, t1, t0
addi t0, x0, 0
add  t0, t0, t1
lw   t2, 0(t0)
sw   t2, -152(x8)


lw    t0, -152(x8)


addi x2, x2, -4
sw    t0, 0(x2)


addi t0, x0, 3


sw    t0, -156(x8)


lw    t1, -156(x8)


addi t0, x0, 2
sll  t1, t1, t0
addi t0, x0, 0
add  t0, t0, t1
lw   t2, 0(t0)
sw   t2, -160(x8)

```



```
lw    t0, -160(x8)
```

```
addi x2, x2, -4
```

```
sw    t0, 0(x2)
```

```
addi x2, x2, -4
```

```
sw    x8, 0(x2)
```

```
addi x2, x2, -4
```

```
sw    x1, 0(x2)
```

```
jal   x1, diferente
```

```
lw    x1, 0(x2)
```

```
lw    x8, 4(x2)
```

```
addi x2, x2, 8
```

```
addi x2, x2, 8
```

```
sw    x10, -164(x8)
```

```
lw    t0, -164(x8)
```

```
sw    t0, -168(x8)
```

```
lw    t0, -44(x8)
```

```
sw    t0, -172(x8)
```

```
lw    t0, -172(x8)
```

```
addi x2, x2, -4
```

```
sw    t0, 0(x2)
```

```
addi t0, x0, 5
```

```
sw    t0, -176(x8)
```

```
lw    t0, -176(x8)
```

```
addi x2, x2, -4
```

```
sw    t0, 0(x2)
```

```
addi x2, x2, -4
```

```
sw    x8, 0(x2)
```

```
addi x2, x2, -4
```

```
sw    x1, 0(x2)
```

```
jal   x1, resto
```

```
lw    x1, 0(x2)
```

```
lw    x8, 4(x2)
```

```
addi x2, x2, 8
```

```
addi x2, x2, 8
```

```
sw    x10, -180(x8)
```

```
lw    t0, -180(x8)
```

```
sw    t0, -184(x8)
```

```
lw    t0, -44(x8)
```

```
sw    t0, -188(x8)
```

```
lw    t0, -188(x8)
```

```
addi x2, x2, -4
```

```
sw    t0, 0(x2)
```

```
lw    t0, -60(x8)
```

```
sw    t0, -192(x8)
```

```
lw    t0, -192(x8)
```

```
addi x2, x2, -4
```

```
sw    t0, 0(x2)
```

```
addi x2, x2, -4
```

```
sw    x8, 0(x2)
```

```
addi x2, x2, -4
```

```
sw    x1, 0(x2)
```

```
jal   x1, mdc
```

```
lw    x1, 0(x2)
```

```
lw    x8, 4(x2)
```

```
addi x2, x2, 8
```

```
addi x2, x2, 8
```

```
sw    x10, -196(x8)
```

```
lw    t0, -196(x8)
```

```
sw    t0, -200(x8)
```

```
addi t0, x0, 0
```

```
sw    t0, -204(x8)
```

```
lw    t1, -204(x8)
```

```
addi t0, x0, 2
```

```
sll  t1, t1, t0
```

```
addi t0, x0, 24
```

```
add  t0, t0, t1
```

```
lw    t2, 0(t0)
```

```
sw    t2, -208(x8)
```

```
lw    t0, -208(x8)
```

```
sw    t0, -212(x8)
```

```
addi t0, x0, 5
```

```
sw    t0, -216(x8)
```

```
lw    t1, -216(x8)
```

```
addi t0, x0, 2
```

```
sll  t1, t1, t0
```

```
addi t0, x0, 24
```

```
add  t0, t0, t1
```

```
lw    t2, 0(t0)
```

```
sw    t2, -220(x8)
```

```
lw    t0, -220(x8)
```

```
sw    t0, -224(x8)
```

```
lw    t0, -44(x8)
```

```
sw    t0, -228(x8)
```

```
lw    t0, -228(x8)
```

```
addi x2, x2, -4
```

```
sw    t0, 0(x2)
```

```
lw    t0, 0(x2)
```

```
addi x2, x2, 4
```

```
sw    t0, 2040(x0)
```

```
lw    t0, -60(x8)
```

```
sw    t0, -232(x8)
```

```
lw    t0, -232(x8)
```

```
addi x2, x2, -4
```

```
sw    t0, 0(x2)
```

```
lw    t0, 0(x2)
```

```
addi x2, x2, 4
```

```
sw    t0, 2040(x0)
```

```
lw    t0, -84(x8)

sw    t0, -236(x8)

lw    t0, -236(x8)

addi  x2, x2, -4
sw    t0, 0(x2)

lw    t0, 0(x2)
addi  x2, x2, 4
sw    t0, 2040(x0)

lw    t0, -108(x8)

sw    t0, -240(x8)

lw    t0, -240(x8)

addi  x2, x2, -4
sw    t0, 0(x2)

lw    t0, 0(x2)
addi  x2, x2, 4
sw    t0, 2040(x0)

lw    t0, -120(x8)
```

```
sw    t0, -244(x8)
```

```
lw    t0, -244(x8)
```

```
addi x2, x2, -4
```

```
sw    t0, 0(x2)
```

```
lw    t0, 0(x2)
```

```
addi x2, x2, 4
```

```
sw    t0, 2040(x0)
```

```
lw    t0, -144(x8)
```

```
sw    t0, -248(x8)
```

```
lw    t0, -248(x8)
```

```
addi x2, x2, -4
```

```
sw    t0, 0(x2)
```

```
lw    t0, 0(x2)
```

```
addi x2, x2, 4
```

```
sw    t0, 2040(x0)
```

```
lw    t0, -168(x8)
```

```
sw    t0, -252(x8)
```

```
lw    t0, -252(x8)
```

```
addi x2, x2, -4
sw    t0, 0(x2)
```

```
lw     t0, 0(x2)
addi x2, x2, 4
sw     t0, 2040(x0)
```

```
lw     t0, -184(x8)
```

```
sw     t0, -256(x8)
```

```
lw     t0, -256(x8)
```

```
addi x2, x2, -4
sw     t0, 0(x2)
```

```
lw     t0, 0(x2)
addi x2, x2, 4
sw     t0, 2040(x0)
```

```
lw     t0, -200(x8)
```

```
sw     t0, -260(x8)
```

```
lw     t0, -260(x8)
```

```
addi x2, x2, -4
sw     t0, 0(x2)
```



```
lw    t0, 0(x2)
addi  x2, x2, 4
sw    t0, 2040(x0)
```

```
lw    t0, -212(x8)
```

```
sw    t0, -264(x8)
```

```
lw    t0, -264(x8)
```

```
addi  x2, x2, -4
sw    t0, 0(x2)
```

```
lw    t0, 0(x2)
addi  x2, x2, 4
sw    t0, 2040(x0)
```

```
lw    t0, -224(x8)
```

```
sw    t0, -268(x8)
```

```
lw    t0, -268(x8)
```

```
addi  x2, x2, -4
sw    t0, 0(x2)
```

```
lw    t0, 0(x2)
addi  x2, x2, 4
sw    t0, 2040(x0)
```

```
# ---- fim de main ----  
    addi x2, x8, 0  
    jalr x0, x1, 0
```

A.3.3 Código Executável (hex) Completo

```
7ff00113  
7ff10113  
7ff10113  
7ff10113  
011000ef  
0000006f  
00010433  
fec10113  
00c42283  
fe542e23  
00842283  
fe542c23  
ffc42283  
ff842303  
0062c3b3  
0013a393  
fe742a23  
ff442283  
00028e63  
00100293  
fe542823  
ff042503  
00040113  
00008067  
0180006f  
00000293  
fe542623  
fec42503  
00040113  
00008067  
00040113  
00008067  
00010433  
fec10113  
00c42283
```

fe542e23
00842283
fe542c23
ffc42283
ff842303
0062c3b3
0013a393
0013c393
fe742a23
ff442283
00028e63
00100293
fe542823
ff042503
00040113
00008067
0180006f
00000293
fe542623
fec42503
00040113
00008067
00040113
00008067
00010433
fec10113
00c42283
fe542e23
00842283
fe542c23
ffc42283
ff842303
005323b3
fe742a23
ff442283
00028e63
00c42283
fe542823
ff042503
00040113
00008067

0180006f
00842283
fe542623
fec42503
00040113
00008067
00040113
00008067
00010433
fdc10113
00c42283
fe542e23
00842283
fe542c23
ffc42283
ff842303
0062a3b3
0013c393
fe742a23
ff442283
02028a63
00c42283
fe542823
00842283
fe542623
ff042283
fec42303
406283b3
fe742423
fe842503
00040113
00008067
0300006f
00842283
fe542223
00c42283
fe542023
fe442283
fe042303
406283b3
fc742e23

fdc42503
00040113
00008067
00040113
00008067
00010433
fcc10113
00842283
fe542e23
00100293
fe542c23
ffc42283
ff842303
005323b3
0013c393
fe742a23
ff442283
00028e63
00842283
fe542823
ff042503
00040113
00008067
0c80006f
00842283
fe542623
00100293
fe542423
fec42283
fe842303
406283b3
fe742223
fe442283
ffc10113
00512023
ffc10113
00812023
ffc10113
00112023
f79ff0ef
00012083

00412403
00810113
00410113
fea42023
00842283
fc542e23
00200293
fc542c23
fdc42283
fd842303
406283b3
fc742a23
fd442283
ffc10113
00512023
ffc10113
00812023
ffc10113
00112023
f25ff0ef
00012083
00412403
00810113
00410113
fca42823
fe042283
fd042303
006283b3
fc742623
fcc42503
00040113
00008067
00040113
00008067
00010433
fd410113
00c42283
fe542e23
00842283
fe542c23
ffc42283

ff842303
0262c3b3
fe742a23
ff442283
fe542823
00c42283
fe542623
ff042283
fe542423
00842283
fe542223
fe842283
fe442303
026283b3
fe742023
fec42283
fe042303
406283b3
fc742e23
fdc42283
fc542c23
fd842283
fc542a23
fd442503
00040113
00008067
00040113
00008067
00010433
fd810113
00842283
fe542e23
00000293
fe542c23
ffc42283
ff842303
0062c3b3
0013a393
0013c393
fe742a23
ff442283

08028063
00c42283
fe542823
ff042283
ffc10113
00512023
00842283
fe542623
fec42283
ffc10113
00512023
ffc10113
00812023
ffc10113
00112023
f05ff0ef
00012083
00412403
00810113
00810113
fea42423
fe842283
fe542223
00842283
fe542023
fe042283
00542623
fe442283
fc542e23
fdc42283
00542423
f59ff06f
00c42283
fc542c23
fd842503
00040113
00008067
00040113
00008067
00010433
fd810113

00000293	
fe542e23	
ffc42283	
fe542c23	
ff842283	
fe542a23	
00842283	
fe542823	
ff442283	
ff042303	
0062a3b3	
fe742623	
fec42283	
04028e63	
ff842283	
fe542423	
7fc02283	
fe542223	
fe842303	
00200293	
00531333	
00c42283	
006282b3	
fe442383	
0072a023	
ff842283	
fe542023	
00100293	
fc542e23	
fe042283	
fdc42303	
006283b3	
fc742c23	
fd842283	
fe542c23	
f85ff06f	
00040113	
00008067	
00010433	
fbcb10113	
00000293	

fe542e23
ffc42283
fe542c23
ff842283
fe542a23
00842283
fe542823
ff442283
ff042303
0062a3b3
fe742623
fec42283
0a028c63
00842283
fe542423
00100293
fe542223
fe842283
fe442303
406283b3
fe742023
ff842283
fc542e23
fe042283
fdc42303
406283b3
fc742c23
fd842283
fc542a23
ff842283
fc542823
fd442283
fc542623
fcc42303
00200293
00531333
01042283
006282b3
0002a383
fc742423
fd042303

00200293
00531333
00c42283
006282b3
fc842383
0072a023
ff842283
fc542223
00100293
fc542023
fc442283
fc042303
006283b3
fa742e23
fbc42283
fe542c23
f29ff06f
00040113
00008067
00010433
fc410113
00000293
fe542e23
ffc42283
fe542c23
00000293
fe542a23
ff442283
fe542823
ff842283
fe542623
00842283
fe542423
fec42283
fe842303
0062a3b3
fe742223
fe442283
06028a63
ff042283
fe542023

ff842283
fc542e23
fdc42303
00200293
00531333
00c42283
006282b3
0002a383
fc742c23
fe042283
fd842303
006283b3
fc742a23
fd442283
fe542823
ff842283
fc542823
00100293
fc542623
fd042283
fcc42303
006283b3
fc742423
fc842283
fe542c23
f6dff06f
ff042283
fc542223
fc442503
00040113
00008067
00040113
00008067
00010433
fa810113
00000293
fe542e23
ffc42283
fe542c23
00100293
fe542a23

ff442283
fe542823
ff842283
fe542623
00842283
fe542423
fec42283
fe842303
0062a3b3
fe742223
fe442283
0e028263
ff842283
fe542023
fe042303
00200293
00531333
00c42283
006282b3
0002a383
fc742e23
00000293
fc542c23
fdc42283
fd842303
005323b3
fc742a23
fd442283
04028663
ff042283
fc542823
ff842283
fc542623
fcc42303
00200293
00531333
00c42283
006282b3
0002a383
fc742423
fd042283

fc842303
026283b3
fc742223
fc442283
fe542823
02c0006f
ff042283
fc542023
00100293
fa542e23
fc042283
fbc42303
026283b3
fa742c23
fb842283
fe542823
ff842283
fa542a23
00100293
fa542823
fb442283
fb042303
006283b3
fa742623
fac42283
fe542c23
efdff06f
ff042283
fa542423
fa842503
00040113
00008067
00040113
00008067
00010433
ef410113
00000293
fe542e23
ffc42283
ffc10113
00512023

00600293	
fe542c23	
ff842283	
ffc10113	
00512023	
ffc10113	
00812023	
ffc10113	
00112023	
bfdff0ef	
00012083	
00412403	
00810113	
00810113	
fea42a23	
00000293	
fe542823	
ff042283	
ffc10113	
00512023	
01800293	
fe542623	
fec42283	
ffc10113	
00512023	
00600293	
fe542423	
fe842283	
ffc10113	
00512023	
ffc10113	
00812023	
ffc10113	
00112023	
c39ff0ef	
00012083	
00412403	
00810113	
00c10113	
fea42223	
00000293	

fe542023
fe042283
ffc10113
00512023
00600293
fc542e23
fdc42283
ffc10113
00512023
ffc10113
00812023
ffc10113
00112023
ce5ff0ef
00012083
00412403
00810113
00810113
fca42c23
fd842283
fc542a23
00000293
fc542823
fd042283
ffc10113
00512023
00600293
fc542623
fcc42283
ffc10113
00512023
ffc10113
00812023
ffc10113
00112023
d69ff0ef
00012083
00412403
00810113
00810113
fca42423

fc842283
fc542223
00000293
fc542023
fc042303
00200293
00531333
00000293
006282b3
0002a383
fa742e23
fbc42283
ffc10113
00512023
00500293
fa542c23
fb842303
00200293
00531333
00000293
006282b3
0002a383
fa742a23
fb442283
ffc10113
00512023
ffc10113
00812023
ffc10113
00112023
ef0ff0ef
00012083
00412403
00810113
00810113
faa42823
fb042283
fa542623
00100293
fa542423
fa842303

00200293
00531333
00000293
006282b3
0002a383
fa742223
fa442283
ffc10113
00512023
00400293
fa542023
fa042303
00200293
00531333
00000293
006282b3
0002a383
f8742e23
f9c42283
ffc10113
00512023
ffc10113
00812023
ffc10113
00112023
ec4ff0ef
00012083
00412403
00810113
00810113
f8a42c23
f9842283
f8542a23
00600293
f8542823
f9042283
ffc10113
00512023
ffc10113
00812023
ffc10113

00112023	
f18ff0ef	
00012083	
00412403	
00810113	
00410113	
f8a42623	
f8c42283	
f8542423	
00200293	
f8542223	
f8442303	
00200293	
00531333	
00000293	
006282b3	
0002a383	
f8742023	
f8042283	
ffc10113	
00512023	
00300293	
f6542e23	
f7c42303	
00200293	
00531333	
00000293	
006282b3	
0002a383	
f6742c23	
f7842283	
ffc10113	
00512023	
ffc10113	
00812023	
ffc10113	
00112023	
cb8ff0ef	
00012083	
00412403	
00810113	

00810113
f6a42a23
f7442283
f6542823
00200293
f6542623
f6c42303
00200293
00531333
00000293
006282b3
0002a383
f6742423
f6842283
ffc10113
00512023
00300293
f6542223
f6442303
00200293
00531333
00000293
006282b3
0002a383
f6742023
f6042283
ffc10113
00512023
ffc10113
00812023
ffc10113
00112023
c90ff0ef
00012083
00412403
00810113
00810113
f4a42e23
f5c42283
f4542c23
fd442283

f4542a23
f5442283
ffc10113
00512023
00500293
f4542823
f5042283
ffc10113
00512023
ffc10113
00812023
ffc10113
00112023
eb8ff0ef
00012083
00412403
00810113
00810113
f4a42623
f4c42283
f4542423
fd442283
f4542223
f4442283
ffc10113
00512023
fc442283
f4542023
f4042283
ffc10113
00512023
ffc10113
00812023
ffc10113
00112023
eecff0ef
00012083
00412403
00810113
00810113
f2a42e23

f3c42283	
f2542c23	
00000293	
f2542a23	
f3442303	
00200293	
00531333	
01800293	
006282b3	
0002a383	
f2742823	
f3042283	
f2542623	
00500293	
f2542423	
f2842303	
00200293	
00531333	
01800293	
006282b3	
0002a383	
f2742223	
f2442283	
f2542023	
fd442283	
f0542e23	
f1c42283	
ffc10113	
00512023	
00012283	
00410113	
7e502c23	
fc442283	
f0542c23	
f1842283	
ffc10113	
00512023	
00012283	
00410113	
7e502c23	
fac42283	

f0542a23
f1442283
ffc10113
00512023
00012283
00410113
7e502c23
f9442283
f0542823
f1042283
ffc10113
00512023
00012283
00410113
7e502c23
f8842283
f0542623
f0c42283
ffc10113
00512023
00012283
00410113
7e502c23
f7042283
f0542423
f0842283
ffc10113
00512023
00012283
00410113
7e502c23
f5842283
f0542223
f0442283
ffc10113
00512023
00012283
00410113
7e502c23
f4842283
f0542023

f0042283
ffc10113
00512023
00012283
00410113
7e502c23
f3842283
ee542e23
efc42283
ffc10113
00512023
00012283
00410113
7e502c23
f2c42283
ee542c23
ef842283
ffc10113
00512023
00012283
00410113
7e502c23
f2042283
ee542a23
ef442283
ffc10113
00512023
00012283
00410113
7e502c23
00040113
00008067